



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației



PROIECT DE DIPLOMĂ

Coordonator științific:
conf. dr. ing. Cristian Nicolae
Buțincu

Absolvent:
Alexandru Găină

Iași, 2026



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației



Platformă Cloud de Prevenție a Atacurilor DDoS

PROIECT DE DIPLOMĂ

Coordonator științific:
conf. dr. ing. Cristian Nicolae
Buțincu

Absolvent:
Alexandru Găină

Iași, 2026

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII PROIECTULUI DE DIPLOMĂ

Subsemnatul GĂINĂ ALEXANDRU,
legitimat cu CI seria XV nr. 820850, CNP 5030514330215
autorul lucrării PLATFORMĂ CLOUD DE PREVENȚIE A ATA CURILOR
DDoS

elaborată în vederea susținerii examenului de finalizare a studiilor de licență, programul de studii TEHNOLOGIA INFORMAȚIEI organizat de către Facultatea de Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi” din Iași, sesiunea IULIE a anului universitar 2025-2026, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 - Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a convențiilor internaționale privind drepturile de autor.

Data

25.06.2026

Semnătura



Cuprins

1	Introducere	1
1.1	Motivație	1
1.2	Obiectivele generale ale lucrării	1
1.3	Metodologia și instrumentele folosite	2
1.4	Structura lucrării	2
2	Fundamentarea teoretică și documentarea bibliografică	3
2.1	Conceptele fundamentale ale atacurilor DDoS	3
2.1.1	Atacuri de tip Volumetric și de Amplificare	3
2.1.2	Atacuri la nivelul aplicației (L7 OSI)	3
2.1.3	Atenuarea atacurilor la intrarea în infrastructura de protecție (On Ramp)	4
2.2	Standarde IETF (RFC) aplicate în ecosistemele proiectului	4
2.2.1	Securitatea DNS și procesarea interogărilor	4
2.2.2	Inspecția WAF și normalizarea traficului	4
2.2.3	Retenția CDN și disponibilitatea resurselor	5
2.2.4	Trasabilitatea IP-ului în arhitecturi proxy	5
2.3	Analiza comparativă a arhitecturilor din industrie	5
2.3.1	Absorbția atacurilor prin rutare Anycast	6
2.3.2	Load balancing distribuit pe pools și endpoints	6
2.3.3	Decuplarea stării și arhitectura stateless	6
2.4	Identificarea problemelor și elaborarea specificațiilor soluției propuse	7
2.4.1	Identificarea problemelor în soluțiile existente	7
2.4.2	Elaborarea specificațiilor platformei propuse	7
3	Soluția propusă	9
3.1	Reiterarea problemei, strategia de rezolvare și cerințele sistemului	9
3.1.1	Reiterarea problemei	9
3.1.2	Strategia de rezolvare și abordări originale	9
3.1.3	Cerințele sistemului (User Requirements)	9
3.2	Alegerea tehnologiilor, a platformei de execuție și analiza hardware	10
3.2.1	Analiza platformei de execuție și a infrastructurii hardware	10
3.2.2	Alegerea limbajului de programare și a bibliotecilor	10
3.2.3	Alegerea bazei de date	11
3.2.4	Arhitectura generală a sistemului și pipeline-ul de procesare	12
3.3	Proiectarea și implementarea modulelor software	14
3.3.1	Modulul DNS Resolver	15
3.3.1.1	Ecosistemul DNS și Rutarea Inteligentă	15
3.3.1.2	Algoritmul Resolverului Iterativ și politicile de Caching	15
3.3.1.3	Mecanisme defensive împotriva atacurilor volumetrice	17
3.3.1.4	Control Plane și Load Balancing dinamic pe nodurile active	18

3.3.2	Modulul PoP (Load Balancer L7)	18
3.3.2.1	Rolul Point of Presence (PoP) și Preluarea Traficului (On Ramp)	18
3.3.2.2	Mecanismul de Health-Check și Descoperirea Dinamică a Nodurilor	19
3.3.2.3	Echilibrarea încărcării pe Endpoints (Algoritmul Round-Robin)	20
3.3.2.4	Securizarea firelor de execuție împotriva atacurilor Slowloris	20
3.3.2.5	Rutarea Proxy și Transparența Traficului	21
3.3.3	Modulul WAF	21
3.3.3.1	Arhitectura și Integrarea în Ecosistem	21
3.3.3.2	Prevenirea Atacurilor Volumetrice la Nivel HTTP (Rate Limiting)	21
3.3.3.3	Inspecția Traficului, Normalizarea și Detecția Bazată pe Semnături	22
3.3.3.4	Redirecționarea cererii către Origine (Off Ramp) și Interfața cu Rețeaua CDN	22
3.3.4	Modulul CDN	25
3.3.4.1	Content Delivery Network (CDN): Retenție, Caching și Revalidare	25
3.3.4.2	Evaluarea Directivelor de Control și Calculul Valabilității (Freshness)	25
3.3.4.3	Arhitectura Cheilor și Gestionarea Conținutului Dinamic prin antetul Vary	25
3.3.4.4	Revalidarea Condiționată și Protecția la Cache Poisoning (Freshening)	25
4	Testarea aplicației și rezultate experimentale	27
4.1	Punerea în funcțiune și configurarea mediului de test	27
4.2	Testarea scalabilității și a mecanismelor de Load Balancing	27
4.2.1	Distribuția traficului L3/L4 la nivel global	27
4.2.2	Distribuția traficului on ramp L7 la nivel de PoP	28
4.3	Testarea funcțională a securității: WAF și protecția volumetrică	29
4.3.1	Protecția la nivelul Resolver-ului DNS (L3/L4)	29
4.3.2	Inspecția profundă a traficului On Ramp (L7 Signatures)	29
4.3.3	Impactul WAF - CDN în atenuarea atacurilor HTTP Flood	30
4.4	Fiabilitate, Failover și Limitările sistemului	31
4.4.1	Recuperarea la căderea totală a pool-ului WAF (Failover)	31
4.4.2	Eficiența mecanismului de detecție (Health Checks)	32
4.4.3	Limitări arhitecturale	32
4.5	Evaluarea performanței End-to-End (E2E)	33
4.5.1	Analiza Eficienței Globale (Concluzia experimentelor)	34
5	Concluzii	35
5.1	Realizarea obiectivelor și contribuții personale	35
5.2	Comparație cu alte proiecte și soluții similare	35
5.3	Posibile direcții de dezvoltare	36
5.4	Lecții învățate pe parcursul dezvoltării proiectului de diplomă	36
	Bibliografie	37
	Anexe	39
1	Topologia Rețelei și Serviciile Containerizate din DNS	39
2	Codul funcției get_nearest_ancestor	42
3	Codul modulului de identificare a PoP-urilor active	42
4	Topologia Rețelei și Serviciile Containerizate din PoP	43
5	Codul funcției de semnalare în platformă a PoP-urilor active	44
6	Codul clasei responsabile de redirectionare corectă a cererilor către WAFs	44

7	Codul funcției responsabile de înregistrarea WAF în rândul nodurilor active	45
8	Dictionarul de semnături Regex	46

Platformă Cloud de Prevenție a Atacurilor DDoS

Alexandru Găină

Rezumat

În era digitală curentă, disponibilitatea serviciilor web este vitală, atacurile cibernetice de tip DDoS reprezentând o amenințare critică. Motivația acestei lucrări constă în nevoia unor sisteme de apărare scalabile și transparente, capabile să asigure o izolare eficientă a serverelor de origine împotriva întreruperilor serviciilor generate de fluxuri de cereri fictive.

Obiectivul principal al tezei este dezvoltarea și testarea unei platforme cloud modulare destinate prevenirii atacurilor DDoS și optimizării livrării de conținut web.

Metodologia a presupus proiectarea unei arhitecturi bazate pe containere. Soluția propusă integrează un modul DNS Resolver responsabil de load balancing global pe pool-uri (L3/L4 OSI) și Puncte de Prezență (PoP) care preiau traficul de intrare inițiat de clienți, definit drept flux on ramp. Filtrarea la nivelul aplicației este realizată de instanțe WAF (Web Application Firewall) cu rol în inspecția detaliată a pachetelor și limitarea fluxului. Acestea sunt susținute de o rețea CDN și o bază de date Redis pentru gestionarea politicilor de retenție a datelor și distribuția sarcinilor prin load balancing pe endpoints. După procesare, cererile legitime care nu sunt deservite din memoria cache sunt redirectionate în siguranță către serverul de origine, formând traficul de ieșire din platformă, definit drept flux de off ramp.

Evaluările experimentale End-to-End confirmă atingerea obiectivelor. Rezultatele demonstrează o sinergie excelentă între modulele WAF și CDN, platforma reușind să blocheze eficient atacurile volumetrice și semantice, păstrând în același timp un nivel ridicat de access concurrent pentru utilizatorii legitimi. În concluzie, sistemul asigură failover rapid, o rutare off ramp sigură și o vizibilitate deplină asupra traficului.

Cuvinte-cheie: Cloud, Prevenție DDoS, WAF, CDN, Load balancing, DNS Resolver.

Capitolul 1. Introducere

1.1. Motivație

Proiectul se încadrează în domeniile sistemelor distribuite și al securității cibernetice.

În cadrul proiectului se va implementa o platformă cloud de prevenție împotriva atacurilor Distributed Denial-of-Service (DDoS).

O introducere pentru acest proiect poate fi constituită chiar de alegerea temei. Internetul, ca și infrastructură, a căpătat o amploare atât de mare, datorită ușurinței cu care omul a ajuns să aibă acces la acesta, încât foarte multe societăți economice și-au mutat activitatea în online. De la trusturi media de știri, conținut audio-video prin intermediul platformelor de streaming, aplicații de banking, health, social-media, ride-sharing, cumpărături, jocuri, cursuri ș.a.m.d.

Astfel observăm faptul că necesitatea disponibilității serviciilor online a devenit o nevoie critică. Economia, dar și confortul individual al oamenilor depind în prezent de ecosisteme digitale complexe. Utilizatorii se așteaptă la acces/răspunsuri instantanee, la orice moment neîntrerupt de timp la date, ceea ce forțează furnizorii de servicii să gestioneze volume din ce în ce mai mari și mai imprevizibile de trafic.

Această dependență majoră creează o vulnerabilitate sistemică. Resursele web sunt expuse public atât traficului organic, cât și atacurilor volumetrice cu cereri valide sau invalide din punct de vedere sintactic, dar nejustificate de o nevoie reală din partea unui utilizator legitim. Definim astfel un atac Distributed Denial of Service (DDoS) ca traficul sintetic generat de botnets/scripturi sau alte instrumente logice de injecție cu trafic malițios de cereri către o resursă expusă la nivel web cu scopul de a epuiza resursele de memorie/calcul ale serverelor de origine și scoaterea serviciului din funcțiune sau cel puțin limitarea promptitudinii răspunsurilor la cererile clienților legitimi.

Ținând cont de nevoia providerilor de servicii de a-și menține activitatea în online, capacitatea de a absorbi, filtra și direcționa traficul devine obiectul platformei propuse în continuare împotriva atacurilor DDoS.

1.2. Obiectivele generale ale lucrării

Scopul principal al acestui proiect este dezvoltarea și implementarea unei platforme cloud robuste pentru prevenția atacurilor DDoS, menținând în același timp o rată optimă de livrare a conținutului legitim din partea serverelor origin. Pentru a atinge acest obiectiv, au fost stabilite următoarele obiective:

- Proiectarea unei arhitecturi scalabile, care să ofere unui orchestrator instrumentele necesare adaptării puterii de calcul/procesare în funcție de traficul de moment
- Integrarea mecanismelor de logging capabile să asigure transparența deciziilor luate de fiecare modul instanțiat, în timp real
- Proiectarea și implementarea unui DNS Resolver care să itereze prin DNS Name Domains, să asocieze domeniile căutate de utilizatorii primari ai internetului cu adresele IP ale serverelor platformei a cărei temă face obiectul acestei documentații
- Implementarea unui Web Application Firewall (WAF) capabil să detecteze activitățile malițioase și să blocheze atacurile la nivel de aplicație
- Proiectarea și implementarea unui Content Delivery Network (CDN) capabil să absoarbă traficul prin servirea resurselor stocate în cache
- Definirea politicilor de data retention în cadrul cache-ului distribuit al CDN

1.3. Metodologia și instrumentele folosite

În vederea atingerii obiectivelor propuse, arhitectura platformei a fost inspirată de topologiile de rețelistică și proiectare documentate la nivelul liderilor din industria de securitate cloud, precum Cloudflare, adaptată și construită în manieră proprie.

La nivel modular, dezvoltarea componentelor logice a respectat cu strictețe standardele IETF (Internet Engineering Task Force), asigurând compatibilitatea sistemului cu arhitectura globală a internetului. Astfel, logica de interogare și comunicarea inter-noduri pentru componenta DNS Resolver au fost implementate respectând protocoalele descrise în RFC 1034[1] și RFC 1035[2].

Pentru inspecția și prelucrarea traficului web la nivelul WAF-ului, a fost necesară înțelegerea semanticii tranzacțiilor la nivelul protocolului HTTP, implementată pe baza RFC 9110. Mai mult, pentru a preveni tehnicile de evaziune ale atacatorilor, cererile au fost supuse unui proces de normalizare a URI-urilor (decodare și validare sintactică) cerută conform RFC 3986[3], permițând astfel aplicarea corectă a semnăturilor de securitate.

Componenta CDN și logica de stocare au fost modelate urmărind standardele de caching HTTP din RFC 9111[4], iar managementul conținutului expirat a fost integrat conform extensiilor propuse în RFC 5861[5]. Deoarece infrastructura utilizează un sistem de reverse proxy-uri pe mai multe niveluri (de la POP către endpoint-urile WAF și către origini), înlănțuirea și identificarea adreselor IP reale ale clienților a fost menținută pe tot parcursul traficului on ramp și off ramp prin intermediul antetului X-Forwarded-For, procedeu documentat în RFC 7239[6].

1.4. Structura lucrării

1. **Capitolul 1: Introducere** - Motivația alegerii temei, contextul actual al securității web și obiectivele arhitecturii propuse.
2. **Capitolul 2: Fundamentarea teoretică și documentarea bibliografică.**
3. **Capitolul 3: Soluția arhitecturală propusă**
 - **Arhitectura de Bază și Rutarea DNS** - Detalierea structurii generale a platformei, algoritmii folosiți la nivelul Resolverului și în atenuarea atacurilor de tip Flood și Amplification.
 - **Preluarea Traficului și Load Balancing** - Analiza nivelului POP, mecanismele anycast de direcționare a traficului și evaluarea în timp real a stării nodurilor WAF.
 - **Logica de inspecție la nivelul WAF** - Metodologia de validare a traficului la Nivelul 7 OSI, sistemele de rate-limiting și recunoașterea semnăturilor malițioase.
 - **Rețeaua de Caching CDN** - Prezentarea sistemului de caching, managementul invalidărilor și politicile de comunicare cu serverele origin.
4. **Capitolul 4: Rezultate experimentale** - Prezentarea statisticilor înregistrate de platformă în cadrul testelor efectuate pe module.
5. **Capitolul 5: Concluzii** - Susținerea abordării propuse în vederea atingerii scopului propus.

Capitolul 2. Fundamentarea teoretică și documentarea bibliografică

2.1. Conceptele fundamentale ale atacurilor DDoS

Un atac de tip Distributed Denial of Service (DDoS) reprezintă o acțiune malițioasă menită să perturbeze funcționalitatea normală a unui serviciu, server sau rețea prin încărcarea țintei cu un volum de trafic imposibil de procesat.

2.1.1. Atacuri de tip Volumetric și de Amplificare

Scopul principal al unui atac volumetric este saturarea lățimii de bandă a legăturii de date a victimei. O subcategorie deosebit de severă o reprezintă atacurile de tip amplificare, de exemplu DNS Amplification. Acesta exploatează caracteristicile protocolului UDP, în mod special lipsa procesului de „handshake” care ar valida adresa IP sursă.

Într-un atac de tip DNS Amplification, asimetria traficului este exploatată prin trimiterea unei cereri DNS de mici dimensiuni către un DNS Resolver deschis, înlocuind (spoofing) adresa IP sursă cu adresa IP a victimei [7]. De regulă, atacatorul va folosi tipul de interogare ANY pentru a forța extragerea tuturor înregistrărilor disponibile. Resolverul procesează cererea și trimite un pachet de răspuns masiv, cu un factor de amplificare ridicat, direct către victimă. În acest context, rutarea standard nu oferă nicio protecție; filtrarea inteligentă a pachetelor devine imperativă pentru a identifica asimetria de răspuns și a respinge pachetele malițioase înainte de a epuiza resursele rețelei [8].

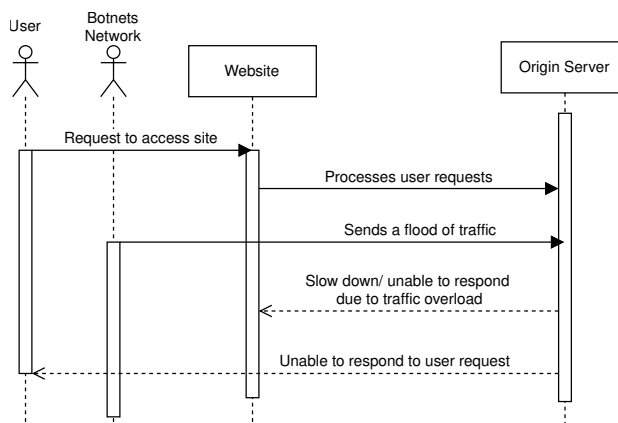


Figura 2.1. Diagrama de secvență a unui atac DDoS

2.1.2. Atacuri la nivelul aplicației (L7 OSI)

Spre deosebire de atacurile volumetrice care vizează nivelurile inferioare ale rețelei, atacurile de la Nivelul 7 OSI [9] țintesc stratul de aplicație (HTTP/HTTPS) cu scopul de a epuiza resursele de calcul ale serverului de origine, precum procesorul sau memoria.

În analiza traficului la acest nivel, este critică diferențierea dintre accesările legitime și un atac de tip HTTP Flood. Un flux legitim de trafic solicită resurse dispersate, o mare parte dintre acestea putând fi reținute și servite direct dintr-un CDN. În schimb, un HTTP Flood este conceput pentru a forța procesări costisitoare pe server [10]. Combaterea acestor vulnerabilități necesită interpunerea unui modul Web Application Firewall (WAF) capabil să extragă antetele HTTP, să normalizeze datele și să evalueze sintaxa fiecărei cereri, aplicând semnături de securitate recunoscute pentru detecția anomaliilor de tip Injection sau Path Traversal [11].

Un alt tip de atac critic este reprezentat de tehnica Slowloris. Spre deosebire de atacurile volumetrice de tip HTTP Flood care încearcă să coplesească rețelele prin forță brută, Slowloris este un

atac de tip "low and slow". Conform documentației de specialitate [12], atacatorul parcurge complet și corect protocolul TCP "3-way handshake" pentru a stabili conexiuni perfect valide, dar speculează modul în care serverele web gestionează sesiunile deschise. Mai exact, acesta menține conexiunile active trimițând anteturi HTTP parțiale și intermitente, fără a finaliza vreodată cererea. Ca rezultat, firele de execuție ale Punctului de Prezență (PoP) rămân blocate în așteptarea pachetelor, ceea ce duce inevitabil la epuizarea resurselor și blocarea traficului on ramp legitim, afectând direct capacitatea de acces concurrent a platformei.

2.1.3. Atenuarea atacurilor la intrarea în infrastructura de protecție (On Ramp)

O abordare deficitară întâlnită în topologiile tradiționale constă în amplasarea mecanismelor de securitate în imediata proximitate a serverului de origine. Dacă un atac volumetric saturează conexiunea fizică a ruterului din data center, aplicația web devine indisponibilă.

Soluția optimă este mutarea protecției la marginea rețelei (Edge Computing). Prin implementarea protocolului ANYCAST, traficul inițial (*on ramp*) este atras și rutat automat către cel mai apropiat PoP (Point of Presence) din punct de vedere topologic [13]. Astfel, forța brută a atacului nu mai este concentrată într-un singur punct. În interiorul acestor puncte de prezență, se aplică un mecanism robust de load balancing, care distribuie echitabil sarcinile către endpoint-urile active. Doar după validarea strictă a cererilor, traficul validat ca fiind legitim își continuă drumul (*off ramp*) către infrastructura protejată.

2.2. Standarde IETF (RFC) aplicate în ecosistemele proiectului

Fundamentarea unei arhitecturi cloud distribuite, integrabile și sigure necesită o aliniere strictă la standardele Internet Engineering Task Force (IETF). Pentru a asigura interoperabilitatea și o bază solidă pentru operațiunile de logare și filtrare (*security and logging mechanisms*), deciziile arhitecturale din spatele platformei au fost suprapuse cu specificațiile Request for Comments (RFC).

2.2.1. Securitatea DNS și procesarea interogărilor

Mecanismele de rezolvare a numelor de domeniu sunt construite după conceptele din RFC 1034 și RFC 1035, care definesc arhitectura ierarhică a spațiului de nume și formatul exact al pachetelor de interogare. Pe baza acestor standarde fundamentale, un DNS Resolver trebuie să fie capabil să proceseze cereri în mod iterativ, navigând prin serverele rădăcină (root) și serverele TLD până la instanțele autoritare. Această arhitectură ierarhică este exploatată pentru a implementa un mecanism de load balancing direct la nivel DNS, prin rotirea dinamică a unor pools de adrese IP ce corespund propriilor PoP-uri (Points of Presence).

Pentru a preveni vulnerabilitățile specifice protocolului UDP, arhitectura încorporează limitările descrise în RFC 8482 [8]. Acest document justifică blocarea sau trunchierea interogărilor de tip ANY (QTYPE 255), eliminând astfel asimetria masivă a traficului care stă la baza atacurilor de tip DNS Amplification.

2.2.2. Inspectia WAF și normalizarea traficului

Filtrarea traficului la Nivelul 7 OSI de către modulul Web Application Firewall (WAF) se fundamentează pe RFC 9110 [14], standardul care definește semantica protocolului HTTP. Înțelegerea detaliată a metodelor cererilor, a codurilor de status și a antetelor (exemplu: User-Agent) este obligatorie pentru a accepta traficul legitim de tip *on ramp* și a respinge anomaliile.

O tehnică majoră de evaziune utilizată de atacatori constă în mascarea semnăturilor malițioase prin encoding. Pentru a preveni acest fenomen, inspectia pachetelor necesită respectarea strictă a RFC 3986 privind sintaxa generică a URI-urilor. Standardul mandatează decodarea (*percent-encoding*) și normalizarea integrală a URI-ului înainte ca acesta să fie evaluat de motorul de securitate. Această etapă anulează posibilitatea evitării filtrelor prin manipulări sintactice în atacurile de tip Path Traversal

sau SQL Injection.

2.2.3. Retenția CDN și disponibilitatea resurselor

Sistemul de distribuție a conținutului (CDN) este proiectat nu doar ca un accelerator de performanță prin livrarea datelor cerute stocate intern, optimizând timpul de răspuns, ci de asemenea ca un mecanism defensiv esențial. Rolul său este de a oferi un nivel ridicat de *acces concurrent*, absorbind vârfurile de trafic la Nivelul 7 și protejând serverele de origine împotriva solicitărilor masive de tip HTTP Flood.

Logica stocării de date venite de la serverul originar în cadrul cache-ului distribuit este construită riguros pe specificațiile RFC 9111 [4]. Acest standard guvernează politicile de caching HTTP, stabilind în primul rând când un răspuns poate fi stocat în condiții de siguranță (evitând reținerea datelor din rute de autentificare sau conținut dinamic marcat cu „no-store”). În al doilea rând, standardul determină prospețimea datelor (*freshness*) pe baza parsării avansate a directivelor Cache-Control (precum max-age sau s-maxage).

Pe lângă regulile de stocare, un aspect fundamental impus de standard este menținerea coerenței datelor (*cache invalidation*). Pentru a preveni servirea de informații inconsistente, logica CDN-ului include invalidarea automată a memoriei. Atunci când un client interacționează cu un endpoint folosind metode HTTP de mutație (*unsafe methods*, precum POST, PUT, PATCH sau DELETE), sistemul detectează modificarea stării și șterge instantaneu înregistrările cache-uite pentru ruta respectivă. Totodată, RFC 9111 [4] impune utilizarea validatorilor condiționali (ETag și Last-Modified). Aceștia permit CDN-ului să valideze cererile recurente ale clienților și să returneze rapid coduri de status 304 (Not Modified) direct de la nivelul PoP-ului, reducând astfel drastic consumul de lățime de bandă în rețeaua *on ramp*.

Pentru a garanta un nivel maxim de *scalabilitate* și continuitate a serviciilor chiar și sub încărcări malițioase, implementarea este completată de extensiile critice definite în RFC 5861. În scenariile în care serverul de origine este ținta unui atac sau procesează greoi cererile pe ruta *off ramp*, introducerea directivelor „stale-while-revalidate” și „stale-if-error” schimbă paradigma de livrare. În loc să mențină conexiunile clienților deschise în așteptare sau să returneze erori de tip Bad Gateway, rețeaua CDN este autorizată să livreze imediat conținut ușor învechit (*stale*) din memorie. În paralel, revalidarea resursei se desfășoară asincron, în fundal. Această decuplare totală a experienței utilizatorului de starea de funcționare a serverului de origine asigură o operabilitate superioară și menține disponibilitatea sistemului în condiții de stres extrem.

2.2.4. Trasabilitatea IP-ului în arhitecturi proxy

Într-o infrastructură care utilizează rutare anycast și reverse proxy-uri pe mai multe niveluri, adresa IP sursă de la nivelul TCP se modifică odată cu preluarea cererii de către nodul de filtrare. Pentru a nu pierde vizibilitatea asupra originii atacatorului în faza de tranziție către traficul *off ramp*, se utilizează standardul RFC 7239. Acesta reglementează folosirea antetului X-Forwarded-For, care extrage și propagă adresa IP a clientului inițial prin toate nivelurile rețelei. Conservarea trasabilității este o condiție obligatorie pentru execuția corectă a algoritmilor de limitare a fluxului (*rate-limiting*) și pentru izolarea endpoint-urilor compromise.

2.3. Analiza comparativă a arhitecturilor din industrie

În domeniul securității cibernetice și al rețelelor de distribuție, arhitecturile dezvoltate de liderii industriei, precum Cloudflare [15], reprezintă standardul de referință. Analiza acestor soluții oferă o perspectivă clară asupra arhitecturilor actuale și a mecanismelor necesare pentru a face față atacurilor DDoS la scară mare. Soluțiile enterprise moderne au abandonat complet dependența de load balancer-ele tradiționale *on-premises*, bazate pe limitări hardware fixe, migrând către infrastructuri cloud-native capabile de o scalare orizontală masivă.

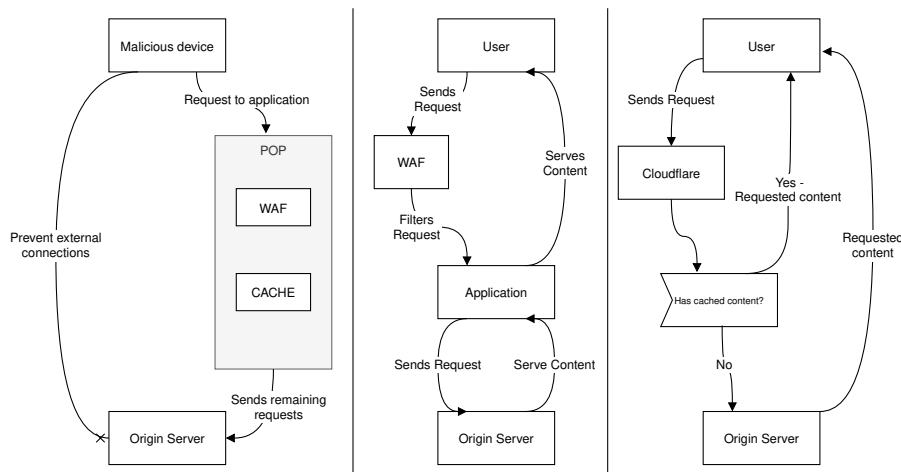


Figura 2.2. Reprezentarea comportamentului (la nivel conceptual) al infrastructurii Cloudflare [16]

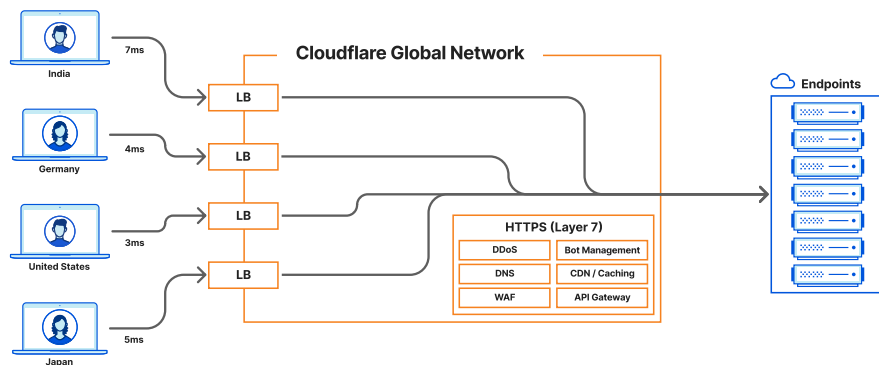


Figura 2.3. Grafic arhitectură Cloudflare Network [16]

2.3.1. Absorbția atacurilor prin rutare Anycast

Mecanismul principal prin care arhitecturile mature asigură conceptul de scalabilitate în fața atacurilor volumetrice este utilizarea protocolului Anycast [17]. În loc să concentreze cererile într-un singur centru de date, platformele atrag traficul *on ramp* direct la intrarea în infrastructura rețelei, direcționându-l întotdeauna către cel mai apropiat PoP din punct de vedere topologic și al latenței. Această dispersie geografică permite rețelei globale să absoarbă atacuri de mari dimensiuni, întrucât forța botnet-ului este divizată între sute de centre de date, împiedicând congestionarea.

2.3.2. Load balancing distribuit pe pools și endpoints

Modul în care traficul este manipulat la nivel de rutare definește capacitatea de reziliență a platformei. Arhitecturile moderne aplică un mecanism avansat de load balancing structurat pe mai multe niveluri. În prima fază, la nivel de DNS Resolver, interogările nu întorc o singură adresă IP, ci liste echilibrate de adrese extrase din pools specifice, distribuind astfel traficul între multiple zone geografice. Ulterior, odată ce pachetele ajung în interiorul unui PoP, ruterul intern distribuie încărcarea către endpoint-urile active (istanțele de filtrare). Decizia de rutare către un endpoint se face doar dacă acesta răspunde pozitiv la monitorizările continue de stare (*health checks*), izolându-se astfel imediat nodurile defecte sau suprasolicitate.

2.3.3. Decuplarea stării și arhitectura stateless

O trăsătură esențială identificată în ecosistemele de top este decuplarea stării globale de logica de execuție. Nodurile WAF și serverele de filtrare sunt complet stateless, independente de celelalte

tranzacții efectuate anterior. Ele nu păstrează local metricile de trafic sau listele IP-urilor blocate, ci se bazează pe baze de date in-memory rapide pentru a valida cererile. Această arhitectură permite ridicarea instantanee de noi containere de procesare pentru a oferi un nivel înalt de procesare a fluxului, permițând acces simultan la politicile de evaluare.

Mai mult, managementul rețelei interne în aceste centre de date moderne favorizează topologiile izolate, cu adresare controlată nativ de platforma cloud, evitând dependențele aduse de un server DHCP tradițional. Prin conectarea acestor componente stateless cu mecanisme de securitate și logging centralizate, operatorii au o capacitate perfectă de urmărire a atacurilor. La finalul acestui proces complex, doar pachetele validate și complet inofensive sunt dirijate *off ramp* către serverele de origine, garantând integritatea aplicațiilor web protejate.

2.4. Identificarea problemelor și elaborarea specificațiilor soluției propuse

2.4.1. Identificarea problemelor în soluțiile existente

Deși soluțiile de clasă enterprise analizate anterior oferă performanțe superioare în atenuarea atacurilor DDoS, ele prezintă anumite limitări din perspectiva implementării în medii private sau în infrastructuri de cercetare academică. Un minus major este determinat de natura lor opacă de tip „black-box”; lipsa controlului direct asupra modului în care algoritmi iau deciziile de filtrare și rutare limitează vizibilitatea și adaptabilitatea sistemului la nevoi specifice.

2.4.2. Elaborarea specificațiilor platformei propuse

Pentru a identifica și răspunde acestor atacuri, platforma proiectată în prezenta lucrare propune o infrastructură cloud transparentă și ușor de supervizat. Pe baza analizei teoretice, s-au stabilit următoarele specificații și caracteristici tehnice așteptate:

- Rutare inteligentă și securitate la nivel DNS: Implementarea unui DNS Resolver iterativ, capabil să parcurgă ierarhia numelor de domeniu (inclusiv interogarea serverelor TLD, de asemenea proiectate și implementate la o scară suficient de mare pentru testare). Acesta trebuie să includă protecții native împotriva atacurilor de amplificare și să asigure un prim strat de load balancing prin rotirea unor grupuri de adrese IP.
- Preluarea traficului la intrarea în rețea: Adoptarea conceptelor de rutare anycast pentru a direcționa traficul *on ramp* către cel mai apropiat PoP. La acest nivel, sarcina va fi distribuită în mod echitabil către endpoint-urile active, izolând automat nodurile defecte.
- Inspecție WAF la Nivelul 7 OSI: Dezvoltarea unui modul capabil să evalueze și să normalizeze pachetele HTTP în timp real, identificând și neutralizând cererile malițioase (precum SQL Injection) și aplicând limitări stricte asupra fluxului înainte ca pachetele să fie trimise *off ramp* către serverele de origine.
- Performanța și retenția datelor: Construirea unui CDN distribuit care să respecte politicile de caching HTTP, reducând încărcarea serverului de origine și garantând un nivel ridicat de *acces concurrent* la nivelul PoP.
- Arhitectură stateless și izolare: Eliminarea dependenței de un server DHCP extern prin containerizarea serviciilor în rețele virtuale izolate. Logica platformei va fi decuplată de memoria sistemului folosind baza de date in-memory Redis, asigurând astfel conceptul de scalabilitate orizontală.
- Monitorizare și vizibilitate: Integrarea unor *security and logging mechanisms* detaliate, care să ofere administratorilor o trasabilitate perfectă asupra întregului ciclu de viață al unui pachet în rețea.

Capitolul 3. Soluția propusă

3.1. Reiterarea problemei, strategia de rezolvare și cerințele sistemului

3.1.1. Reiterarea problemei

În contextul amenințărilor cibernetice actuale, disponibilitatea serviciilor web este constant pusă în pericol de atacuri DDoS din ce în ce mai sofisticate. Problema fundamentală pe care această lucrare o abordează este ineficiența soluțiilor tradiționale, centralizate, în fața atacurilor volumetrice (precum amplificarea DNS) și a traficului malițios la Nivelul 7 OSI. Echipamentele hardware on-premises devin rapid un punct unic de eșec (Single Point of Failure) atunci când lățimea de bandă a routerului de intrare este saturată. Rutarea standard și load balancer-ele clasice nu pot face diferența între traficul legitim și un atac de tip HTTP Flood, permițând pachetelor să epuizeze resursele serverului de origine.

3.1.2. Strategia de rezolvare și abordări originale

Pentru a rezolva această problemă, strategia adoptată constă în dezvoltarea unei platforme cloud-native, distribuite și scalabile. Abordarea propusă utilizează concepte de Edge Computing, atrăgând traficul on-ramp direct către cel mai apropiat punct de procesare (PoP) al rețelei.

O componentă esențială a acestei strategii o reprezintă decuplarea completă a logicii de filtrare (WAF) și a celei de rutare (DNS Resolver, PoP) de memoria sistemului. Prin integrarea unei baze de date in-memory centralizate, nodurile de procesare devin complet „stateless”. Spre deosebire de sistemele care se bazează pe configurări complexe și pe alocări stricte tip DHCP, această rețea izolată permite distrugerea și recrearea instantanee a containerelor (endpoints) după necesitate. Astfel, prin combinarea unui load balancing avansat la nivel DNS cu distribuția locală a traficului la nivel de PoP, atacul este atenuat prin dispersie, iar către serverele de origine se trimite exclusiv trafic validat pe ruta off-ramp.

3.1.3. Cerințele sistemului (User Requirements)

Pentru ca platforma să aducă o valoare reală și să poată fi integrată în scenarii operaționale, dezvoltarea aplicației a urmărit respectarea unui set strict de cerințe:

- **Protecția clientului fără a afecta utilizatorul legitim:** Procesul de preluare, parsare a pachetelor, interogare a sistemului CDN și livrare a răspunsului trebuie să aibă o latență minimă, fără a afecta experiența utilizatorului final.
- **Disponibilitate ridicată (Acces concurent):** Arhitectura trebuie să suporte accesări simultane masive. Implementarea necesită multiplexare eficientă la nivel de socket și un mecanism avansat de caching pentru a absorbi cererile repetitive fără a contacta repetat originea.
- **Portabilitate și independență hardware:** Sistemul trebuie să poată fi rulat și testat pe orice platformă (Linux, Windows, macOS) fără modificări ale codului sursă, necesitând un mediu de execuție containerizat și rețele virtuale definite software.
- **Monitorizare (Security and logging mechanisms):** Platforma trebuie să asigure un mecanism detaliat de detecție și trasabilitate. Menținerea vizibilității asupra IP-ului sursă real este o cerință obligatorie pentru luarea corectă a deciziilor de banare și pentru raportarea securității în interiorul clusterului.

3.2. Alegerea tehnologiilor, a platformei de execuție și analiza hardware

Transpunerea modelului teoretic într-o soluție funcțională a impus o proiectare atentă a ecosistemului tehnologic. Alegerea mediului de execuție și a echipamentelor software a fost rezultatul unei analize strategice, având ca obiectiv atingerea unui echilibru optim între viteza de procesare a traficului, modularitatea codului și capacitatea de extindere dinamică a infrastructurii.

3.2.1. Analiza platformei de execuție și a infrastructurii hardware

Sistemele tradiționale de protecție DDoS rulează de obicei pe echipamente hardware dedicate (bare-metal), optimizate cu acceleratoare de rețea. Totuși, pentru o soluție cloud-native flexibilă, dependența de un anumit tip de hardware sau de un sistem de operare gazdă reprezintă un dezavantaj major.

Astfel, platforma de execuție aleasă pentru acest proiect este Docker[18]. Prin utilizarea containerizării la nivel de sistem de operare, am abstractizat complet platforma hardware adiacentă. Această abordare prezintă următoarele avantaje:

- **Izolarea rețelei:** Serviciile (DNS Resolver, WAF, CDN) rulează în rețele virtuale de tip bridge definite prin software (cu ajutorul fișierelor docker-compose). Această configurație asigură o adresare IP internă stabilă, fără blocajele sau latențele introduse de un server DHCP fizic extern.
- **Scalabilitate orizontală:** Pe aceeași platformă hardware, fie că este un laptop sau o mașină virtuală în cloud, multiplicarea nodurilor de procesare (instance endpoints) se face alocând secvențial resurse RAM și CPU dinamic, fără a opri serviciile deja funcționale.

3.2.2. Alegerea limbajului de programare și a bibliotecilor

Logica computațională a fost implementată folosind limbajul Python. Deși limbaje compilate precum C sau Go oferă un avantaj brut la nivel de procesor, Python a fost preferat pentru flexibilitatea superioară în manipularea șirurilor de caractere (esențială pentru evaluarea regex-urilor OWASP) și pentru ecosistemul vast de instrumente dedicate rețelilor. Platforma este puternic orientată spre operațiuni de tip I/O (Input/Output). Prin utilizarea pachetelor native de threading, Python asigură un nivel înalt de acces concurent, permițând procesarea în paralel a conexiunilor la nivelul fiecărui PoP.

Pentru a păstra controlul absolut asupra arhitecturii și a evita dependențele inutile, dezvoltarea a integrat o serie de biblioteci, aplicate în funcție de necesitățile specifice ale fiecărui mediu:

- **Manipularea pachetelor DNS:** În mediul DNS, biblioteca `dnslib` a fost esențială pentru parșurarea la nivel de byte a pachetelor UDP/TCP. Aceasta permite decodarea interogărilor clienților și construirea arhitecturii de răspuns (inclusiv asignarea adreselor pentru mecanismul de load balancing pe pools).
- **Decuplarea stării și configurări:** În întregul sistem (DNS, PoP, WAF, CDN), interfața `redis` a asigurat comunicarea ultra-rapidă cu baza de date in-memory, susținând logica stateless a platformei. Gestiunea sigură a adreselor, porturilor și a credențialelor a fost realizată prin `python-dotenv`, care previne expunerea datelor sensibile (hardcoding) în codul sursă.
- **Comunicare de rețea și simulare:** Biblioteca nativă `socket` a fost utilizată atât pentru arhitectura de interceptare a pachetelor, cât și pentru dezvoltarea unui server de origine (origin server) controlat, necesar în faza de testare. Suplimentar, în mediul WAF/PoP, biblioteca `requests` a fost folosită pentru a simula cereri HTTP complexe, testând astfel robustețea filtrelor.
- **Măsurători și vizualizarea performanțelor:** Evaluarea sistemului a necesitat colectarea unor metrici precise. Biblioteca `time` a fost folosită pentru calculul latenței, al limitei de rată și al politicilor de data retention. Pentru transpunerea acestor date în informații vizuale (grafice de

performanță), s-au folosit numpy și matplotlib, elemente de bază în analiza comportamentului sistemului în scenariile de încărcare simulată.

3.2.3. Alegerea bazei de date

Într-o arhitectură cloud-native distribuită, cuplarea stării aplicației de instanța de procesare (de exemplu, stocarea contoarelor de trafic sau a sesiunilor pe discul local al unui nod WAF) reprezintă o deficiență majoră de proiectare. O astfel de abordare anulează direct eficiența mecanismelor de load balancing; dacă un client abuziv este redirecționat prin algoritmul round-robin pe trei noduri WAF diferite în decursul a câteva secunde, niciunul dintre noduri nu va înregistra independent încălcarea pragului de limitare a ratei.

Pentru a depăși această limitare și a garanta un sistem fail-safe, s-a optat pentru decuplarea totală a stării proceselor (stateless architecture) prin integrarea Redis[19], o bază de date avansată in-memory. Operațiunile efectuate exclusiv în memoria RAM elimină blocajele clasice de I/O ale discurilor fizice, oferind o latență constantă de ordinul sub-milisecundelor. Astfel, clusterul Redis devine unitatea de stocare a informațiilor principală apelată (single source of truth) pentru întreaga platformă, fiind tehnologia optimă pentru susținerea a trei blocuri operaționale critice:

1. Sincronizarea politicilor de Securitate (Rate-Limiting Global): Pentru ca platforma să intercepteze și să neutralizeze atacurile volumetrice on ramp în timp real, dicționarul de limitare a ratei și lista IP-urilor compromise trebuie să fie partajate la nivelul întregului sistem. Orice decizie de securitate generată de un nod WAF este înscrisă direct în Redis, devenind instantaneu aplicabilă pentru toate celelalte containere din rețea.
2. Managementul Topologiei Dinamice (Service Discovery): Fiind implementată pe bază de containere, platforma operează cu resurse de procesare distribuită. Redis menține o evidență strictă a topologiei rețelei, stocând listele de endpoints funcționale. Prin intermediul thread-urilor dedicate de health-check de la nivelul PoP, starea containerelor este actualizată continuu, asigurând rutarea traficului exclusiv către nodurile perfect capabile să proceseze date.
3. Susținerea Modulului CDN (Data Retention): Pentru a garanta un nivel superior de acces concurrent și a proteja serverul de origine, datele inspectate și validate sunt plasate temporar în memoria Redis. Acest mecanism de cache respectă cu exactitate politicile de freshness impuse prin standardul RFC 9111[4], permițând eliberarea imediată a răspunsurilor HTTP fără a reevalua rutele off ramp costisitoare.

3.2.4. Arhitectura generală a sistemului și pipeline-ul de procesare

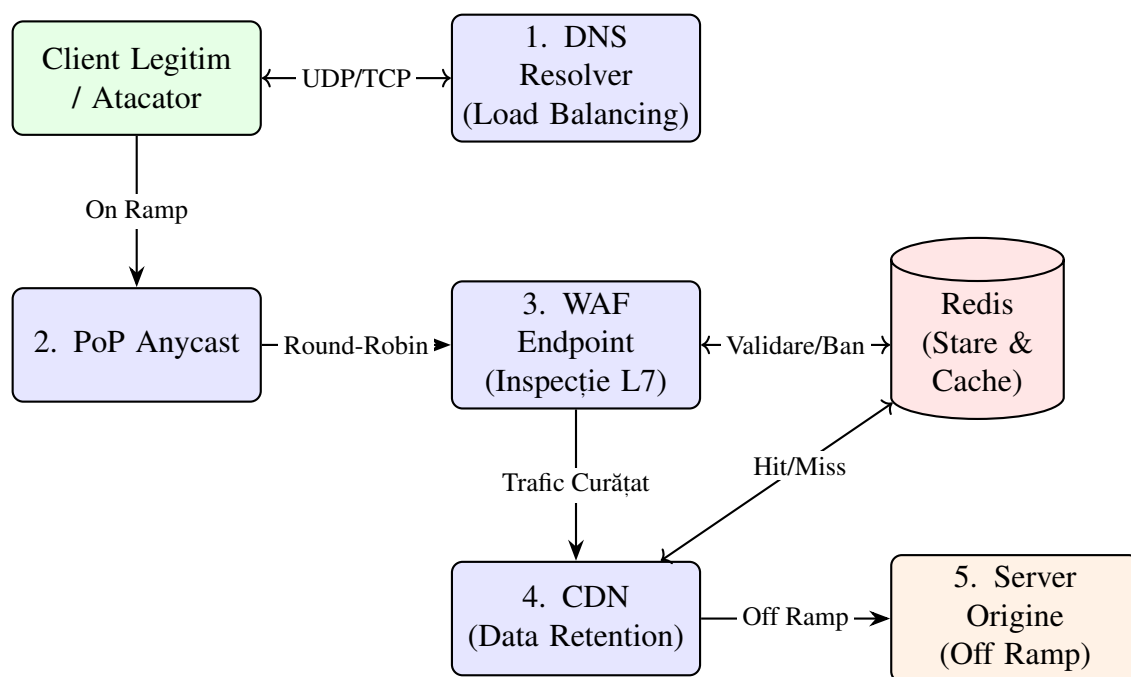


Figura 3.1. Reprezentarea vizuală a pipeline-ului de procesare a traficului și interacțiunea modulelor

Pentru a asigura o protecție eficientă împotriva atacurilor DDoS, platforma a fost proiectată sub forma unui pipeline de filtrare secvențială pe mai multe niveluri. Arhitectura sistemului previne congestionarea serverului de origine abordând atât atacurile volumetrice (la nivel de rețea), cât și pe cele de tip HTTP Flood sau Injection (la nivel de aplicație).

Fluxul de procesare a traficului urmează un traseu strict, împărțit în cinci etape esențiale, de la inițierea cererii până la livrarea conținutului:

1. Interogarea numelui domeniului și Load Balancing la nivel DNS

Orice interacțiune a unui client legitim sau malițios începe cu rezolvarea numelui domeniului căutat. Cererea UDP/TCP este interceptată de DNS Resolver-ul platformei noastre. Acesta parcurge arborele DNS (navigând prin serverele rădăcină și TLD ori până găsește sau nu domeniul căutat) într-o manieră iterativă. În această etapă, platforma blochează interogările anormale (precum cererile ANY) pentru a preveni atacurile de amplificare. Dacă cererea este validă, resolverul nu returnează o singură adresă IP, ci aplică un mecanism de load balancing primar: extrage o adresă disponibilă dintr-un sistem de pools asociate domeniului respectiv și o returnează clientului, dispersând astfel încărcarea încă de la începutul pipeline-ului.

2. Rutarea Anycast și preluarea traficului On Ramp

Odată ce clientul primește adresa IP rezolvată, inițiază conexiunea TCP. Prin utilizarea conceptelor de Anycast, infrastructura cloud atrage acest trafic On Ramp și îl dirijează automat către cel mai apropiat PoP (point of presence) activ din punct de vedere topologic. Această preluare la intrarea în infrastructura rețelei asigură absorbția atacurilor volumetrice masive, forțând dispersia botnet-urilor în mai multe centre de date și împiedicând concentrarea atacului asupra unei singure legături fizice.

3. Distribuția internă pe Endpoints (Load Balancer în PoP)

La intrarea în rețeaua virtuală a unui PoP, traficul este preluat de un ruter software intern. Aici, arhitectura beneficiază de rețele Docker izolate, eliminând întârzierile și vulnerabilitățile aduse

de un server DHCP tradițional. Ruterul PoP interoghează constant baza de date in-memory (Redis) pentru a obține o hartă în timp real a topologiei. Printr-un mecanism automat de health-check, acesta selectează doar nodurile active (endpoints) și distribuie încărcarea în mod round-robin, asigurând o paralelizare perfectă a efortului de procesare.

4. Inspecția de securitate WAF

Pachetul ajunge la un endpoint WAF specific pentru inspecția la Nivelul 7 OSI. În această fază, nodul extrage adresa IP reală a clientului din antetul X-Forwarded-For și verifică imediat în Redis dacă sursa a depășit limita de cereri acceptate în fereastra de timp sau un ban global activ. Dacă IP-ul este curat, WAF-ul normalizează URI-ul și aplică regulile de securitate (semnăturile OWASP) pentru a detecta anomalii de sintaxă. Orice trafic malițios este abandonat (rejected) pe loc, iar sistemul este actualizat prin mecanismele de securitate, completate de log-uri.

5. CDN Data Retention și Dirijarea Off Ramp

Dacă cererea HTTP este considerată legitimă și inofensivă, aceasta intră în procesarea modulului CDN. Sistemul verifică în Redis politicile de data retention și freshness (valabilitatea cache-ului).

- **Cache Hit:** Dacă resursa a fost deja stocată și este considerată de actualitate, CDN-ul livrează răspunsul instantaneu către client, generând răspunsul cu datele din baza de date cu zero efort din partea serverului de origine.
- **Cache Miss:** Dacă resursa nu este în cache (sau trebuie revalidată), nodul de filtrare deschide o conexiune sigură către adevăratul server al clientului (originea). Acest trafic curățat reprezintă ruta off-ramp. Serverul de origine procesează cererea neștiind de atac, trimite datele înapoi la nod, acesta le cache-uește conform politicilor și apoi le livrează clientului final.

3.3. Proiectarea și implementarea modulelor software

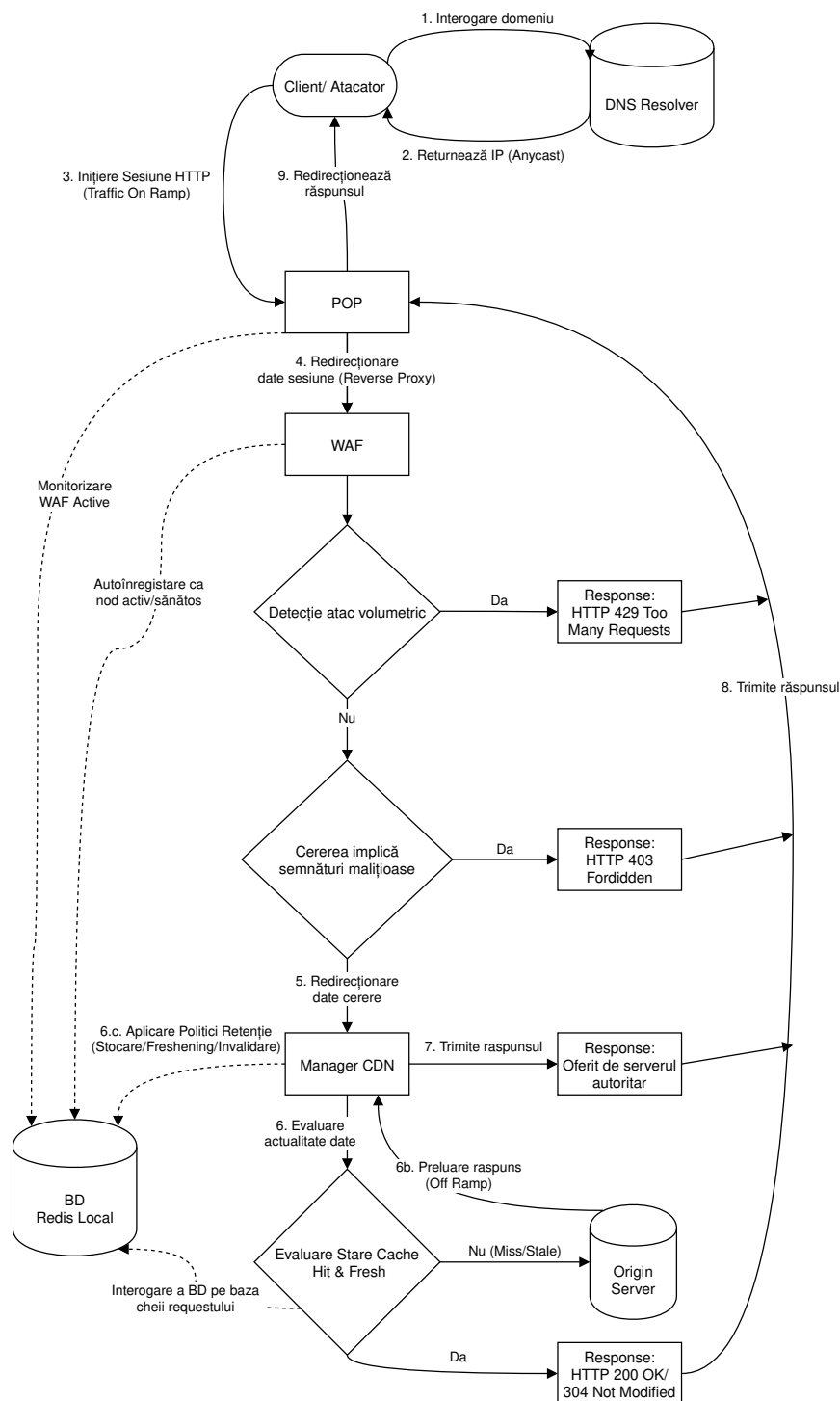


Figura 3.2. Diagrama de flux a întregii infrastructuri

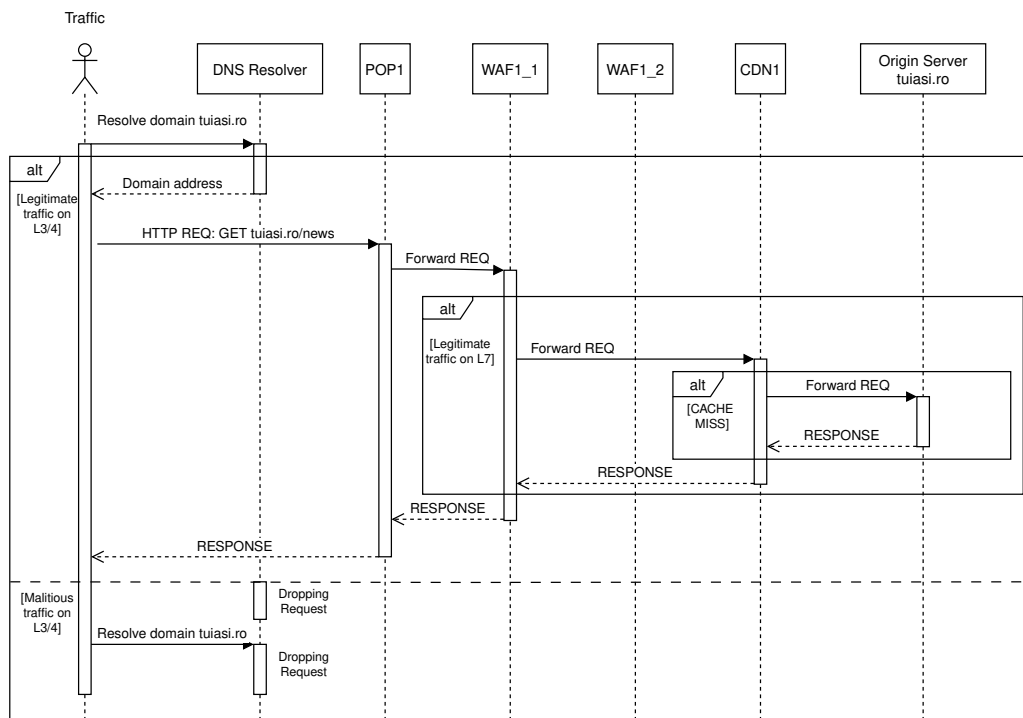


Figura 3.3. Diagrama de secvență a întregii infrastructuri

3.3.1. Modulul DNS Resolver

3.3.1.1. Ecosistemul DNS și Rutarea Inteligentă

Arhitectura de rezolvare a numelor domeniilor a fost proiectată pentru a simula cât mai bine ierarhia reală a internetului, asigurând un mediu de testare izolat, dar complet funcțional din punct de vedere al delegărilor de domenii. Prin intermediul mediului containerizat (Docker), au fost instanțiate trei niveluri distincte de servere autoritare (Name Servers): serverul Root (ns_root), serverul TLD pentru zona „.ro” (ns_rotld) și serverul autoritar propriu (ns_mycloud).

Configurația de bază a acestor containere și a rețelei virtuale izolate poate fi studiată în fișierul de topologie atașat în Anexa 1.

Interacțiunea dintre aceste entități și înregistrarea noilor domenii este gestionată printr-un set de API-uri REST (serviciile rotld_api și mycloud_api). Această decizie arhitecturală permite simularea dinamică a procesului prin care un client își delegă domeniul existent în infrastructura de protecție.

3.3.1.2. Algoritmul Resolverului Iterativ și politicile de Caching

Nucleul acestui modul este reprezentat de un DNS Resolver iterativ dezvoltat în Python. Pentru a garanta un nivel maxim de autenticitate și interoperabilitate cu rețelele externe, comunicarea dintre resolver și Name Servers respectă riguros arhitectura tranzacțională definită în standardele IETF. Prin utilizarea bibliotecii dnslib, interogările și răspunsurile sunt procesate sub formă de obiecte DNS (DNS Query Objects) care reflectă atât convențiile de adresare din RFC 1034, cât și implementarea strictă la nivel de pachet (wire format) stipulată în RFC 1035.

Fiecare pachet transportat prin protocolul UDP este structurat în secțiunile normative: Header, Question, Answer, Authority și Additional. Algoritmul extrage datele necesare rutării analizând flag-urile din Header (precum indicatorul AA - Authoritative Answer) și evaluează delegările extrgând adresele IP ascunse sub formă de Glue Records din secțiunea Additional.

Procesul decizional începe prin interogarea memoriei cache in-memory din Redis. Algoritmul verifică într-o primă fază existența unui răspuns negativ de tip NXDOMAIN stocat anterior sau a unei

înregistrări directe de tip A. În caz de cache miss, înainte de a iniția o parcurgere oarbă a întregului arbore de la serverul Root, algoritmul optimizează traseul apelând funcția de căutare a celui mai apropiat subdomeniu (*get_nearest_ancestor*). Vezi Anexa 2

Această funcție descompune domeniul căutat în etichete (*labels*) și interoghează iterativ baza de date de la cel mai specific nivel către cel mai general pentru a găsi o delegare de tip NS cunoscută (de exemplu, pentru interogarea "*www.mycloud.ro.*", sistemul va căuta delegări succesive pentru "*mycloud.ro.*", apoi "*ro.*", apoi root ". "). Dacă un domeniu superior este găsit, căutarea pornește de la acea adresă, scurtând masiv latența cererii. În cazul în care sistemul nu găsește nicio referință în cache, algoritmul apelează la o structură de siguranță de tip Safety Belt (SBelt), extrăgând adresa IP hardcodată a serverului Root pentru a începe rezolvarea de la zero.

Parcurgerea iterativă propriu-zisă este limitată în mod deliberat la un număr maxim de 10 hop-uri pentru a preveni buclele de rutare. O particularitate avansată a algoritmului implementat o reprezintă tratarea răspunsurilor autoritare volatile. Dacă serverul autoritar returnează o înregistrare valabilă însă valoarea câmpului TTL este mai mică sau egală cu 1, informația nu mai este stocată în cache, ci este expedită direct pentru a preveni alterarea consistenței datelor.

Algoritmul 3.1 Algoritmul de Interogare Iterativă și Caching

```

1: procedure INTEROGAREITERATIVA(qname)
2:   if INCACHE(NXDOMAIN: +qname) then
3:     return null                                ▷ Domeniu validat anterior ca inexistent
4:   pool,ttl ← APLICABALANCING(qname)
5:   if pool ≠ null then
6:     return pool,ttl                             ▷ Cache Hit pentru inregistrari tip A
7:   SLIST ← GETNEARESTANCESTOR(qname)
8:   hops ← 0
9:   while hops < 10 and NOTEMPTY(SLIST) do
10:    hops ← hops + 1
11:    targetIP ← SLIST[0]
12:    raspuns ← INTEROGHEAZANS(targetIP,qname)
13:    if raspuns.AA == 1 then                      ▷ Răspuns Autoritar
14:      if raspuns.TTL ≤ 1 then
15:        return EXTRAGEIP(raspuns), raspuns.TTL  ▷ Date volatile, nu stocăm in cache
16:      SALVEAZAINCACHE(raspuns)
17:      return APLICABALANCING(qname)
18:    else if AREDELEGARI(raspuns) then           ▷ Referral către altă zonă
19:      newSLIST ← EXTRAGEGLUE RECORDS(raspuns)
20:      if NOTEMPTY(newSLIST) then
21:        SLIST ← newSLIST
22:        SALVEAZADELEGAREINCACHE(raspuns)
23:      else
24:        ELIMINAPRIMULELEMENT(SLIST)           ▷ Referral fără înregistrări alipite
25:      else
26:        ELIMINAPRIMULELEMENT(SLIST)           ▷ Adresă invalidă sau timeout
27:      SALVEAZANXDOMAIN(qname)
28:      return null

```

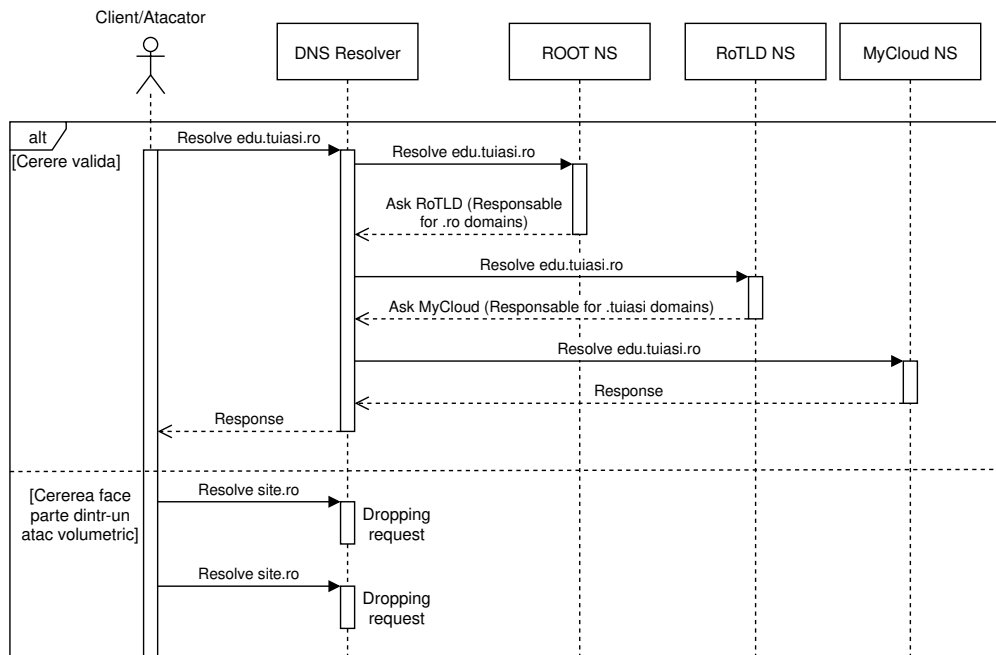


Figura 3.4. Diagrama de secvență: Fluxul de comunicare și delegare între client, resolver și ierarhia DNS

3.3.1.3. Mecanisme defensive împotriva atacurilor volumetrice

Pentru a securiza infrastructura on ramp de la nivelul portului 53, resolverul rulează în interiorul unui server UDP multi-threaded, oferind acces concurrent. Înainte de a consuma resurse parsând obiectul DNS, sistemul aplică o verificare strictă împotriva atacurilor de tip flood.

Pentru a garanta suportul pentru un volum masiv de conexiuni simultane fără întârzieri de rețea, evaluarea IP-urilor se face atomic. Modulul minimizează numărul de accesări ale bazei de date grupând operațiile folosind metoda `pipeline()` din Redis pentru a trimite comenzile de incrementare a contorului și preluare a expirării (`ttl`) într-un singur apel de rețea. Un client malițios primește un ban automat la nivelul întregului sistem dacă respectă următoarea inegalitate matematică:

$$Req_{client} > DNS_FLOOD_MAX_REQS$$

evaluată strict pentru o fereastră temporală definită ca $\Delta t = DNS_FLOOD_WINDOW$.

Mai mult, algoritmul împiedică atacurile de tip DNS Amplification printr-o analiză de conținut a pachetului. Cererile care solicită toate înregistrările disponibile (marcate prin flag-ul `QTYPE = 255`, aferent tipului ANY) sunt respinse imediat, întorcând un răspuns cu antetul `RCODE 5 (REFUSED)`, anulând astfel asimetria masivă a răspunsului specifică acestui tip de atac.

Algoritm 3.2 Mecanismul de Rate Limiting și Prevenire DNS Amplification

```

1: procedure HANDLEUDPREQUEST(pachet, ipClient)
2:   if INCACHE(ban: +ipClient) then
3:     Drop pachet ▷ Blocare silențioasă la nivel L4
4:   Atomic Pipeline (Redis):
5:     cereri ← INCREMENT(req: +ipClient)
6:     tth ← GETTTL(req: +ipClient)
7:     if cereri == 1 or tth ≤ 0 then
8:       SETEXPIRE(req: +ipClient, FLOOD_WINDOW)
9:     if cereri > FLOOD_MAX_REQS then
10:      CACHESET(ban: +ipClient, 1, ex=BAN_TIMEOUT)
11:      Drop pachet
12:     cerereDNS ← PARSEDNS(pachet)
13:     if cerereDNS.qtype == ANY then ▷ Neutralizare amplificare asimetrică
14:       raspuns ← cerereDNS.reply()
15:       raspuns.rcode ← REFUSED
16:       SENDTOCLIENT(raspuns)
17:     return
18:   INTEROGAREITERATIVA(cerereDNS.qname)

```

3.3.1.4. Control Plane și Load Balancing dinamic pe nodurile active

Integrarea nativă a ecosistemului DNS cu restul platformei se realizează prin intermediul unui modul de control `mycloud_control_plane`, a cărui implementare se regăsește în Anexa 3. Acest serviciu acționează ca un mecanism de monitorizare a stării topologiei. Verificând periodic semnalele de heartbeat emise în baza de date de către punctele de prezență (PoP), Control Plane-ul extrage exclusiv adresele IP ale nodurilor active (endpoints) și le injectează în înregistrările de tip A aferente domeniilor protejate.

Această sincronizare dinamică permite aplicarea unui algoritm eficient de load balancing încă de la primul nivel. În momentul formulării unui răspuns DNS afirmativ, nu este livrată o adresă IP unică, ci un *pool* (grup) de adrese. Pentru a dispersa în mod echitabil traficul noilor clienți pe nodurile de tip PoP, resolver-ul realizează o rotație (list slicing) a vectorului de adrese IP returnat. Adresa de start a vectorului este mutată folosind un index aleatoriu R aplicat asupra pool-ului de dimensiune N :

$$Pool_{nou} = \bigcup (Pool[R \dots N - 1], Pool[0 \dots R - 1])$$

Această distribuție uniformă elimină riscul suprasolicitării unei singure rute și oferă o arhitectură eficientă de dispersie a cererilor înregistrate.

3.3.2. Modulul PoP (Load Balancer L7)

3.3.2.1. Rolul Point of Presence (PoP) și Preluarea Traficului (On Ramp)

Modulul PoP (Point of Presence) acționează ca poartă principală de intrare în infrastructura cloud, preluând traficul on ramp direct de la utilizatori, în urma rezolvării adreselor IP executate de către DNS Resolver. Din punct de vedere arhitectural, fiecare PoP este un server proxy L7 (Layer 7) implementat în Python. Pentru a procesa un volum ridicat de cereri simultane fără a bloca execuția (concurrent access support), serverul de bază este instanțiat prin intermediul clasei `ThreadedTCPServer`.

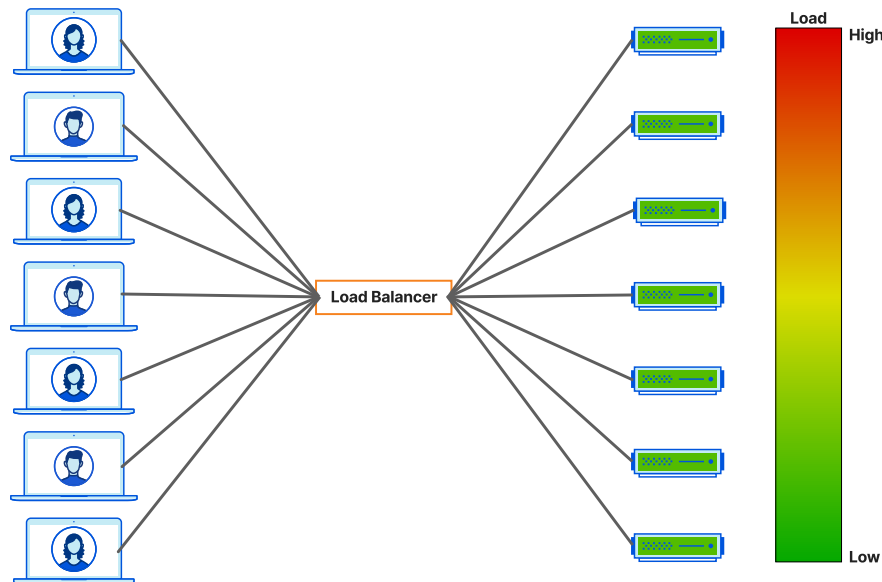


Figura 3.5. Grafic de distribuție a cererilor echitabil între instanțele de procesare [13]

Pentru a menține o sincronizare perfectă între topologia reală și deciziile de rutare superioare, modulul PoP include un proces activ pe tot parcursul rulării serviciului (`send_heartbeat_to_dns`). Acesta trimite periodic un semnal (heartbeats) către instanța globală Redis asociată ecosistemului DNS, confirmând faptul că nodul este capabil să preia trafic.

Acest serviciu vine în completarea modulului din DNS ce citește din aceeași bază de date înregistrările făcute de PoP active. Codul complementar poate fi găsit în Anexa 5.

Conform modului în care a fost proiectat containerul Docker (Anexa 4), fiecare imagine a unui PoP instanțiat poate conține un număr predefinit de instanțe de Web Application Firewall WAF (va fi detaliat în subcapitolul următor).

3.3.2.2. Mecanismul de Health-Check și Descoperirea Dinamică a Nodurilor

Spre deosebire de echilibrarea la nivel DNS, care realizează load balancing pe pools de servere în funcție de încărcarea macro a rețelei, modulul PoP are responsabilitatea de a distribui pachetele la nivel micro, către endpoints (containerele interne care rulează modulele Web Application Firewall).

Pentru a garanta o arhitectură dinamică în care se pot adăuga sau șterge instanțe, sistemul utilizează un mecanism de autodescoperire mediat de Redis. Fiecare endpoint de securitate își semnalează existența în baza de date locală sub forma unui puls prin încărcarea în baza de date a unei chei temporare (`waf_node:*`). PoP-ul monitorizează continuu aceste chei, însă nu rotează traficul aleatoriu către ele.

O componentă separată execută un proces iterativ de health-check asincron: fiecare endpoint din rețea este interogată prin intermediul unei cereri HTTP GET pe ruta `/health`. Doar nodurile care răspund în fereastra de timp alocată cu statusul HTTP 200 (OK) sunt promovate în lista de noduri active. Această filtrare strictă asigură izolarea imediată a containerelor aflate în stare de crash și menținerea integrității sistemului.

Deoarece procesul de monitorizare (health-check) rulează asincron într-un fir de execuție (thread) dedicat în fundal, în timp ce cererile HTTP concurente primite de la clienți extrag și manipulează noduri din aceeași listă, apare o problemă clasică de acces concurent la o resursă partajată. Pentru a garanta consistența datelor (thread-safety) și a preveni excepțiile de tip *race condition*, modificarea vectorului de noduri active a fost izolată în interiorul unei secțiuni critice.

Această secțiune logică este strict protejată de un mecanism de excludere a firelor de execuție la accesul pe o resursă (Mutex Lock) implementat nativ prin biblioteca de sistem (`threading.Lock()`).

Astfel, în momentele în care lista de rute este reconstruită cu nodurile proaspăt validate sau curățată de nodurile moarte, operațiunile de citire și rutare ale serverului proxy sunt temporar suspendate la nivel de thread până la eliberarea resursei. Această tehnică de sincronizare asigură tranziții de stare sigure la nivelul întregului PoP, eliminând complet riscul coruperii memoriei, al apariției indexărilor invalide sau al pierderii de pachete în momentele de încărcare maximă.

Algoritmul 3.3 Algoritmul de Descoperire și Health-Check Asincron

```

1: procedure MONITORIZARENODURIWAF
2:   while true do
3:     endpoints  $\leftarrow$  INTEROGHEAZAREDIS("waf_node:*")
4:     noduriSanatoase  $\leftarrow$   $\emptyset$ 
5:     for each nod in endpoints do
6:       try:
7:         raspuns  $\leftarrow$  HTTPGET(nod + "/health")
8:         if raspuns.status == 200 then
9:           ADAUGALALISTA(noduriSanatoase, nod)
10:        catch Exception:
11:          continue ▷ Nod indisponibil sau in stare de crash
12:        APLICALOCKMUTEX ▷ Intrare în secțiunea critică
13:        listaNoua  $\leftarrow$   $\emptyset$ 
▷ Păstrăm ordinea Round-Robin pentru nodurile deja active
14:        for each nod in endpointsActive do
15:          if nod  $\in$  noduriSanatoase then
16:            ADAUGALALISTA(listaNoua, nod)
▷ Adăugăm nodurile recent descoperite sau recuperate
17:        for each nod in endpoints do
18:          if nod  $\notin$  listaNoua and nod  $\in$  noduriSanatoase then
19:            ADAUGALALISTA(listaNoua, nod)
20:        endpointsActive  $\leftarrow$  listaNoua
21:        ELIBEREAZALOCKMUTEX ▷ Ieșire din secțiunea critică
22:        SLEEP(5 secunde)
  
```

3.3.2.3. Echilibrarea încărcării pe Endpoints (Algoritmul Round-Robin)

Odată ce o tranzacție HTTP/HTTPS este preluată de la un client, decizia de rutare către un anumit nod WAF se face prin aplicarea unui algoritm de load balancing de tip Round-Robin. Logica a fost implementată prin funcția `get_next_endpoint`, care extrage și elimină primul element din lista nodurilor confirmate ca fiind active, reintroducându-l apoi la finalul vectorului.

Deoarece serverul rulează un fir de execuție independent pentru fiecare conexiune client, operațiunea de rotire a listei este securizată matematic prin mecanisme de tip lock (`threading.Lock()`). Astfel, sunt prevenite race conditions (acces simultan la aceeași resursă) în care două thread-uri concurente ar extrage același endpoint simultan.

3.3.2.4. Securizarea firelor de execuție împotriva atacurilor Slowloris

Așa cum s-a detaliat în subcapitolul anterior, serverul alocă un fir de execuție independent pentru fiecare conexiune client în parte. Această arhitectură expune sistemul la vulnerabilități care vizează epuizarea resurselor alocate (thread exhaustion). Un atac critic din această categorie la nivelul stratului aplicație (L7) este reprezentat de tehnica Slowloris.

Pentru a contracara această vulnerabilitate direct la nivelul de preluare a traficului, s-a implementat o politică strictă de întrerupere la nivel de socket (timeout). La inițializarea fiecărei con-

exiuni responsabile cu gestionarea cererilor HTTP, s-a impus o limită absolută de 5 secunde pentru recepționarea integrală a anteturilor. Dacă un client malițios încearcă să țină conexiunea deschisă întârziind transmiterea pachetului complet, socket-ul este închis automat după expirarea acestui interval de grație.

Această abordare eliberează firul de execuție în mod proactiv. Astfel, se asigură faptul că algoritmul de load balancing pe endpoints poate continua neîntrerupt să extragă și să distribuie cererile valide către instanțele WAF, protejând integral disponibilitatea platformei cloud.

Algoritmul 3.4 Algoritmul Round-Robin pentru Rutarea Cererilor L7

```

1: procedure GETNEXTENDPOINT
2:   APLOCKMUTEX                                ▷ Secțiune critică pentru thread-safety
3:   if ISEEMPTY(endpointsActive) then
4:     ELIBEREAZALOCKMUTEX
5:     return null                                ▷ Niciun nod WAF disponibil
6:   endpoint ← EXTRAGEPRIMULELEMENT(endpointsActive)
7:   ADAUGALAFINAL(endpointsActive, endpoint)    ▷ Rotire circulară a listei
8:   ELIBEREAZALOCKMUTEX
9:   return endpoint                                ▷ Returnează nodul către proxy

```

3.3.2.5. Rutarea Proxy și Transparența Traficului

Ultima fază a procesului On Ramp este reprezentată de clasa ProxyHTTPRequestHandler, care reconstruiește și expediază cererea originală către capătul de rețea ales. Pentru a nu compromite deciziile luate la nivelurile următoare, proxy-ul trebuie să asigure un grad înalt de transparență pentru mecanismele de monitorizare și securitate integrate în WAF.

În acest sens, PoP-ul injectează adresa IP originală a clientului în antetul standard X-Forwarded-For. În absența acestui pas crucial, întregul trafic primit de modulele de securitate ar apărea ca provenind de la o singură sursă (adresa IP internă a PoP-ului), fapt ce ar declanșa alarme false și ar sabota mecanismele de rate limiting. Răspunsul întors ulterior de infrastructura cloud (fie de la motorul CDN, fie traficul off ramp adus de la origin server) este transmis înapoi în mod transparent către client, conservând fidel headerele originale și codurile de răspuns aferente stării platformei. Codul aferent poate fi găsit la Anexa 6.

3.3.3. Modulul WAF

3.3.3.1. Arhitectura și Integrarea în Ecosistem

Modulul Web Application Firewall (WAF) reprezintă nucleul de securitate și inspecție al platformei, operând strict la nivelul 7 în arhitectura OSI (Layer 7). Spre deosebire de arhitecturile monolitice în care proxy-ul și firewall-ul sunt strâns cuplate, ecosistemul dezvoltat a fost conceput pentru a oferi o înaltă scalabilitate orizontală. Mai multe instanțe WAF (endpoints) sunt instantiate simultan, independent unele de altele, în spatele modulului PoP.

Pentru a face posibilă integrarea nativă cu mecanismul de health-check descris anterior, fiecare container WAF rulează o rutină de fundal (register_to_redis, Vezi Anexa 7). Acest fir de execuție independent injectează și menține în baza de date locală o cheie temporară cu durată scurtă de viață (TTL). În eventualitatea în care procesul principal al aplicației cedează, cheia expiră în mod natural, iar modulul PoP izolează automat nodul defect, asigurând astfel toleranța la erori a întregii rețele.

3.3.3.2. Prevenirea Atacurilor Volumetrice la Nivel HTTP (Rate Limiting)

Înainte de a supune pachetele unei analize de conținut aprofundate și consumatoare de resurse, modulul WAF implementează un prim filtru defensiv îndreptat împotriva atacurilor de tip HTTP Flood

(DDoS L7). Pentru a asigura corectitudinea mecanismului de penalizare, algoritmul extrage și evaluează adresa IP reală a sursei direct din antetul X-Forwarded-For, antet care este păstrat în siguranță de reverse proxy pe toată durata sesiunii on ramp.

Sistemul utilizează operațiuni atomice prin capabilitatea de pipeline a bazei de date Redis pentru a contoriza cererile în timp real, garantând consistența operațiilor chiar și la niveluri mari de concurență. Dacă rata interogărilor provenite de la un singur utilizator depășește pragul maxim admis ($WAF_FLOOD_MAX_REQS$) în cadrul unei ferestre temporale stricte, accesul adresei IP vizate este revocat automat. Sistemul aplică restricția și returnează codul de eroare HTTP 429 (Too Many Requests), conservând resursele sistemului.

$$\sum_{t=t_0}^{t_0+WAF_FLOOD_WINDOW} Cereri(IP_{sursa}) > WAF_FLOOD_MAX_REQS$$

Algoritmul 3.5 Mecanismul de Rate Limiting la Nivel L7 (WAF)

```

1: procedure CLIENTISRATELIMITED(clientIP)
2:   if INCACHE("ban:" + clientIP) then
3:     return true                                     ▷ Clientul este deja restricționat
4:   cheieRateLimit ← "rate_limit:" + clientIP
5:   Atomic Pipeline (Redis):
6:     cereriCurente ← INCREMENT(cheieRateLimit)
7:     ttlCurent ← GETTTL(cheieRateLimit)
8:     if cereriCurente == 1 or ttlCurent ≤ 0 then
9:       SETEXPIRE(cheieRateLimit, WAF_FLOOD_WINDOW)
10:    if cereriCurente > WAF_FLOOD_MAX_REQS then
11:      CACHESET("ban:" + clientIP, 1, ex=WAF_BAN_TIMEOUT)
12:      DELETECACHE(cheieRateLimit)
13:    return true                                     ▷ Declanșare restricție HTTP 429
14:  return false                                     ▷ Trafic legitim

```

3.3.3.3. Inspecția Traficului, Normalizarea și Detecția Bazată pe Semnături

Traficul validat de mecanismul de rate limiting trece în etapa de inspecție a conținutului (deep packet inspection). Pentru a preveni tehnicile de evaziune utilizate de atacatori, WAF-ul aplică inițial un proces de normalizare asupra obiectului cererii HTTP, decodând caracterele formate URL (percent-encoding conform RFC 3986) atât din calea cererii (path), cât și din corpul acesteia (body).

Payload-ul proaspăt reconstruit, împreună cu antetul User-Agent, sunt evaluate împotriva unui dicționar de semnături structurat sub formă de expresii regulate (Regex). Această abordare este aliniată cu standardele din industrie și recomandările OWASP (Open Worldwide Application Security Project), implementând filtre defensive specifice pentru cele mai critice vulnerabilități web recunoscute la nivel global. Astfel, aceste security and logging mechanisms sunt capabile să identifice rapid tipare malițioase precum SQL Injection (SQLi), Cross-Site Scripting (XSS), Path Traversal sau tentativele de intruziune inițiate de scannere automate și boți (conform CWE-20)[10]. Orice pachet corupt este respins instantaneu către client cu un cod HTTP 403 (Forbidden), interzicându-se accesul către origin server.

Semnăturile propriu-zise pot fi găsite la Anexa 8.

3.3.3.4. Redirecționarea cererii către Origine (Off Ramp) și Interfața cu Rețeaua CDN

Cererile ce trec cu succes de toate filtrele structurale și de securitate trebuie rezolvate și returnate clientului. În această fază, containerul WAF nu mai acționează doar ca un firewall de nivel 7, ci preia rolul unui orchestrator de trafic (reverse proxy intelligent), gestionând comunicarea cu serve-

rele de origine și cu memoria cache locală. Procesul este divizat în patru etape decizionale distincte, implementate în handler-ul principal al serverului:

1. Rezolvarea Domeniului (Global Routing)

Înainte de a iniția vreo conexiune externă, WAF-ul extrage valoarea antetului Host din cererea HTTP și interoghează instanța globală Redis (asociată serviciului de DNS/Control Plane) pentru a determina adresa IP reală a serverului de origine. Dacă domeniul solicitat nu este înregistrat sau rezolvat în platforma MyCloud, tranzacția este oprită imediat, iar sistemul returnează eroarea HTTP 502 (Bad Gateway), prevenind astfel scurgerile de informații sau atacurile de tip SSRF (Server-Side Request Forgery).

2. Evaluarea existenței datelor valide în Cache Local

Odată confirmată ruta, WAF-ul delegă sarcina către managerul rețelei de livrare a conținutului (CDN). Algoritmul extrage o semnătură unică a cererii (construită din adresă, port, cale și header specifice precum Vary) și verifică existența resursei în memoria in-memory Redis aferentă PoP-ului curent. Dacă resursa este găsită (Cache Hit), sistemul optimizează masiv latența:

- Dacă clientul a trimis header de validare (precum If-None-Match pentru ETag), iar resursa locală corespunde, WAF-ul returnează doar statusul HTTP 304 (Not Modified), economisind lățimea de bandă.
- Dacă resursa are încă termenul de valabilitate valid (freshness TTL), aceasta este extrasă complet din memorie și trimisă clientului împreună cu codul HTTP 200.

3. Interogarea Originii (Cache Miss) și Revalidarea

În lipsa unei copii valide în cache, aplicația inițiază traficul off ramp către serverul de origine folosind biblioteca standard `urllib.request`. Pentru a asigura transparența de rețea, headerele originale sunt copiate, iar antetul X-Forwarded-For este injectat cu adresa IP reală a clientului. Mai mult, dacă în cache există o variantă expirată (stale) a resursei, WAF-ul include automat eticheta ETag veche în antetul If-None-Match. Astfel, originii i se oferă șansa de a valida resursa veche cu un răspuns 304, în loc să retransmită întregul conținut (proces denumit freshening).

4. Procesarea Răspunsului și Actualizarea Stării

Răspunsul întors de serverul de origine este interceptat de WAF înainte de a fi expediat clientului. În funcție de codul HTTP primit, aplicația declanșează diverse mecanisme de management al stării:

- Pentru metodele care alterează starea (POST, PUT, DELETE), se apelează procedura de invalidare automată a cache-ului (`invalidate_mutations`).
- Dacă răspunsul îndeplinește condițiile standardelor IETF pentru stocare (evaluat prin funcția `is_cacheable`), acesta este predat modulului CDN pentru a fi indexat și stocat, calculându-se timpii de retenție pe baza directivelor Cache-Control.
- În final, headerele de răspuns, însoțite de marcajul de trasabilitate X-WAF-Node (pentru a identifica în sistemul de load balancing care instanță a procesat cererea), sunt expediate înapoi prin intermediul modulului PoP către utilizatorul final.

Metodele amintite vor fi supuse unei discuții detaliate în subcapitolul următor.

Algoritmul 3.6 Logica de Rutare Off Ramp și Integrarea CDN

```

1: procedure HANDLEOFFRAMP(cerere, clientIP)
2:   originIP ← GETGLOBALREDIS("origin:" + cerere.host)
3:   if originIP == null then
4:     SENDERROR(502, "Bad Gateway: Domeniu nerezolvat in platforma")
5:     return
6:   cheieCache ← CDN.GETREDISKEYS(cerere)
7:   cache ← GETLOCALREDIS(cheieCache)
8:   if cache ≠ null then                                ▷ Evaluarea stratului de memorie locală
9:     if CDN.VALIDATECLIENTREQUEST(cerere.headers, cache) then
10:      SENDRESPONSE(304, cache.headers)                  ▷ Validare cu ETag (Not Modified)
11:      return
12:     if ISFRESH(cache) then
13:       SENDRESPONSE(200, cache.headers, cache.body)
14:       return                                           ▷ Cache Hit - Resursă livrata din memorie
15:   try:                                                 ▷ Cache Miss sau date expirate (Stale)
16:     reqOrigine ← BUILDREQUEST(originIP, cerere)
17:     ADDHEADER(reqOrigine, "X-Forwarded-For", clientIP)
18:
19:   if cache ≠ null and HASETAG(cache) then
20:     ADDHEADER(reqOrigine, "If-None-Match", cache.ETag)
21:
22:   raspuns ← SENDHTTPREQUEST(reqOrigine)
23:
24:   if raspuns.status == 304 and cache ≠ null then
25:     CDN.FRESHENCACHE(cheieCache, raspuns.headers)
26:     SENDRESPONSE(200, cache.headers, cache.body)
27:     return                                           ▷ Freshening - revalidare reușită cu originea
28:
29:   CDN.INVALIDATEMUTATIONS(cerere.method, cheieCache)
30:
31:   if CDN.ISCACHEABLE(cerere.method, raspuns) then
32:     CDN.STORERESPONSE(cheieCache, raspuns)
33:
34:   SENDRESPONSE(raspuns.status, raspuns.headers, raspuns.body)
35:   catch HTTPError as e:
36:     SENDRESPONSE(e.code, e.headers, e.body)
37:   catch Exception:
38:     SENDERROR(502, "Bad Gateway: Serverul de origine nu raspunde")

```

3.3.4. Modulul CDN

3.3.4.1. Content Delivery Network (CDN): Retenție, Caching și Revalidare

Modulul Content Delivery Network (CDN) reprezintă stratul de optimizare a performanței din cadrul platformei, având rolul de a minimiza latența către utilizatorii finali și de a reduce drastic încărcarea pe serverele de origine. Din punct de vedere arhitectural, logica CDN este integrată la nivelul fiecărui nod WAF prin intermediul clasei CDNManager, utilizând baza de date in-memory Redis ca mecanism distribuit de stocare la nivelul fiecărui punct de prezență (PoP).

Funcționalitatea acestui modul a fost proiectată în strictă conformitate cu standardele globale de retenție web, în special RFC 9111 (HTTP Caching) și RFC 5861 (Stale Content).

3.3.4.2. Evaluarea Directivelor de Control și Calculul Valabilității (Freshness)

Înainte ca o resursă să fie considerată eligibilă pentru stocare, sistemul trebuie să decodeze și să interpreteze permisiunile acordate de serverul de origine. Pentru aceasta, aplicația implementează un parser personalizat dedicat antetului Cache-Control. Acesta extrage atât directivele restrictive (precum no-store, private sau must-revalidate), cât și pe cele de temporizare (max-age, s-maxage). O resursă este considerată stocabilă doar dacă întrunește cumulativ o serie de factori de securitate și conformitate:

- Metoda cererii este una safe (GET sau HEAD).
- Codul de răspuns nu reprezintă conținut parțial (se face bypass la HTTP 206 Partial Content).
- Nu există date cu caracter sensibil în tranzacție (se refuză stocarea dacă antetul Authorization este prezent).
- Timpul calculat de expirare (Freshness TTL) este strict pozitiv.

Calculul timpului de valabilitate se realizează în cascadă: directiva s-maxage are prioritate absolută, urmată de max-age. Dacă niciuna nu este specificată, algoritmul recurge la o estimare (fallback) bazată pe antetul clasic Expires, calculând diferența temporală (timestamp) dintre momentul actual și data de expirare furnizată.

3.3.4.3. Arhitectura Cheilor și Gestionarea Conținutului Dinamic prin antetul Vary

O provocare tehnică majoră într-o rețea de tip CDN este stocarea diferitelor versiuni ale aceleiași pagini (de exemplu, variante comprimate GZIP vs. text simplu). Pentru a preveni coruperea conținutului, sistemul implementează o logică de semnare bazată pe antetul Vary.

Stocarea în Redis nu folosește o structură cheie-valoare simplă, ci un tip de date de tip Hash (Dicționar). Cheia principală (Base Key) identifică unic ruta solicitată (ex: cdn:domeniu:port/cale), în timp ce câmpurile Hash-ului (Fields) sunt reprezentate de semnăturile Vary. Semnătura este concatenată dinamic prin evaluarea antetelor cererii clientului, garantând că fiecărei versiuni a resursei îi este alocat un spațiu de memorie independent și sigur. Durata de viață a cheii în Redis (Hard TTL) este calculată adunând timpul de valabilitate (freshness) cu timpul maxim permis de grație setat prin directivele stale-while-revalidate sau stale-if-error.

3.3.4.4. Revalidarea Condiționată și Protecția la Cache Poisoning (Freshening)

Atunci când o resursă stocată în memorie își depășește timpul de valabilitate (devine „stale”), CDN-ul nu o șterge imediat, ci instruieste modulul WAF să execute o tranzacție de revalidare condiționată către serverul de origine, incluzând vechiul marcaj ETag.

Dacă serverul răspunde cu codul HTTP 304 (Not Modified), algoritmul declanșează funcția de Freshening. Aceasta actualizează doar metadatele (noul timestamp și noul TTL permis de antetele

modificate), conservând payload-ul original și economisind timpi masivi de transfer. În acest proces este inclus și un filtru de securitate critic: algoritmul de Freshening ignoră deliberat orice modificare suspectă asupra antetelor care încep cu Content-* la revalidare, o vulnerabilitate des exploatată în atacurile de tip Cache Poisoning.

În plus, mecanismul CDN rămâne sincronizat în permanență cu acțiunile utilizatorului: orice cerere de modificare de tip POST, PUT sau DELETE trimisă pe un anumit endpoint declanșează imediat ștergerea bazei de cache asociate (*invalidate_mutations*), prevenind schimbarea stării aplicației vizate.

Algoritmul 3.7 Logica de Evaluare și Stocare a Conținutului în Rețeaua CDN

```

1: procedure ISCACHEABLE(metoda, reqHeaders, status, respHeaders)
2:   if metoda  $\notin$  {"GET", "HEAD"} then
3:     return false
4:   if status == 206 then
5:     return false ▷ Bypass conținut parțial
6:   if HASKEY(reqHeaders, "Authorization") or HASKEY(respHeaders, "Authorization") then
7:     return false ▷ Bypass protecție date sensibile
8:   cc  $\leftarrow$  PARSECACHECONTROL(respHeaders["Cache-Control"])
9:   if cc.no_store or cc.private then
10:    return false
11:   if CALCULATEFRESHNESS(cc, respHeaders)  $\leq 0$  then
12:    return false
13:   return true

```

```

14: procedure STORERESPONSE(host, port, path, reqHeaders, status, respHeaders, body)
15:   baseKey, varySig  $\leftarrow$  GETREDISKEYS(host, port, path, reqHeaders, respHeaders)
16:   cc  $\leftarrow$  PARSECACHECONTROL(respHeaders["Cache-Control"])
17:   freshnessTTL  $\leftarrow$  CALCULATEFRESHNESS(cc, respHeaders)
18:   safeHeaders  $\leftarrow$  REMOVENOCACHEFIELDS(respHeaders, cc.no_cache_fields)
19:   data  $\leftarrow$  {
20:     status: status,
21:     headers: safeHeaders,
22:     body: body,
23:     freshness_ttl: freshnessTTL
24:   }
25:   maxStale  $\leftarrow$  max(cc.stale_while_revalidate, cc.stale_if_error)
26:   redisHardTTL  $\leftarrow$  freshnessTTL + maxStale
▷ Stocare in structura de tip Dictionar cu suport pentru VARY
27:   REDIS.HSET(baseKey, varySig, JSON(data))
28:   REDIS.EXPIRE(baseKey, redisHardTTL)

```

Capitolul 4. Testarea aplicației și rezultate experimentale

4.1. Punerea în funcțiune și configurarea mediului de test

Platforma cloud dezvoltată pentru prevenția atacurilor DDoS a fost implementată și evaluată într-un mediu virtualizat și strict controlat, utilizând tehnologia de containerizare Docker. Această abordare asigură portabilitatea codului și replicarea exactă a arhitecturii de producție. Punerea în funcțiune a sistemului se realizează în mod automatizat prin intermediul fișierelor de configurare `docker-compose.yml` (Anexa 4), care orchestrează crearea și interconectarea modulelor în rețeaua internă izolată.

Procesul de lansare inițializează următoarele componente:

- **Modulul DNS Resolver:** Acționează ca punct global de intrare, gestionând interogările pentru TLD-urile înregistrate (ex: `edu.tuiasi.ro` sau `api.mycloud.ro`) și rutând traficul prin tehnici specifice către adresele IP.
- **Punctul de Prezență (PoP – point of presence):** Acționează ca un reverse proxy de tip L7, fiind prima entitate care interceptează traficul on ramp generat de clienți.
- **Clusterul Redis Local:** Este inițializat atât ca memorie partajată pentru politicile de cache ale rețelei CDN, cât și ca sistem de stocare ultrarapid pentru contoarele de securitate și mecanismele de health check.
- **Nodurile WAF (Web Application Firewall):** Instanțele de securitate sunt scalate dinamic (de la 1 la n containere) prin parametrul `-scale waf_node` la momentul lansării, demonstrând flexibilitatea arhitecturii.

Fiecare modul a fost instanțiat folosind variabile de mediu specifice pentru a ajusta parametrii de retenție, toleranța la flood și duratele de penalizare (ban timeouts), permițând astfel un control granular asupra fiecărui scenariu de testare.

4.2. Testarea scalabilității și a mecanismelor de Load Balancing

Pentru ca un sistem de prevenție DDoS să fie eficient, acesta trebuie să posede o arhitectură capabilă să absoarbă vârfuri masive de trafic fără a crea blocaje la nivel de rețea (bottlenecks). Acest obiectiv a fost atins prin implementarea a două straturi distincte de distribuire a sarcinii: un nivel global la rezolvarea numelui de domeniu și un nivel regional la preluarea conexiunilor HTTP.

4.2.1. Distribuția traficului L3/L4 la nivel global

Primul experiment a vizat testarea mecanismului de load balancing pe pools, implementat în interiorul modulului DNS Resolver. Pentru a măsura eficiența distribuției, a fost conceput un script de test care a generat un volum de 10.000 de interogări DNS consecutive către adresa resolver-ului, solicitând rezolvarea domeniului țintă. Modulul DNS analizează pool-ul de adrese active (simulând o rețea anycast) și extrage o adresă validă.

Rezultatele experimentale demonstrează că, deși procesul de selecție este stocastic (aleatoriu) pentru a împiedica predictibilitatea în cazul unui atac direcționat, distribuția finală converge către o uniformitate perfectă datorită legii numerelor mari. Niciun IP din pool-ul Punctelor de Prezență nu a fost suprasolicitat, demonstrând o scalabilitate orizontală robustă.

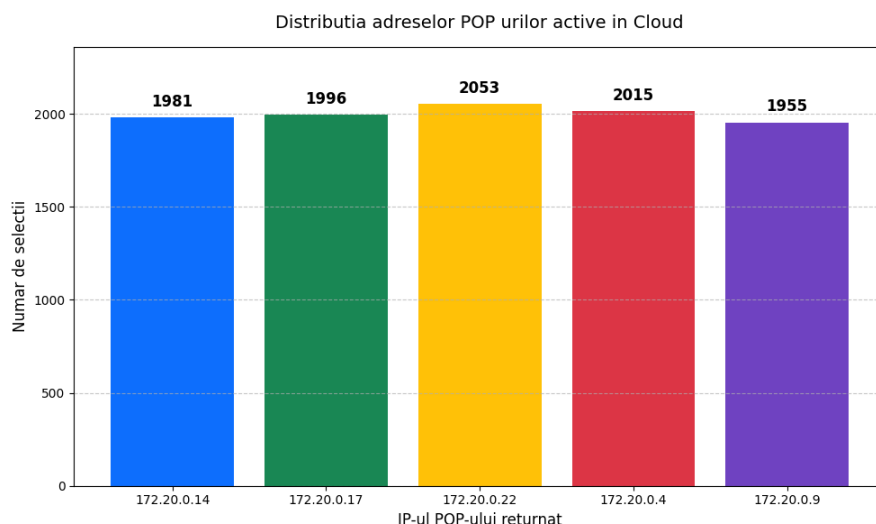


Figura 4.1. Grafic de distribuție a cererilor între instanțele PoP

4.2.2. Distribuția traficului on ramp L7 la nivel de PoP

Al doilea experiment s-a concentrat pe mecanismul de load balancing pe endpoints, operat direct de către PoP. Odată ce traficul on ramp e primit la nivelul PoP-ului, acesta trebuie distribuit către containerele WAF active din rețea folosind un algoritm Round-Robin strict.

Pentru acest test au fost emise 3000 de cereri HTTP concurente către PoP. O etapă critică în configurarea acestui experiment a fost relaxarea temporară a filtrelor de protecție din interiorul nodurilor WAF (mărirea ferestrei de evaluare și a pragului maxim de cereri). Fără acest artificiu de configurare, WAF-ul ar fi clasificat propriu test de load balancing drept un atac volumetric și ar fi început să returneze erori de tip 429 Too Many Requests, viciind datele.

Trasabilitatea cererilor a fost asigurată prin analiza antetului personalizat X-WAF-Node, inserat de fiecare container în răspunsul returnat.

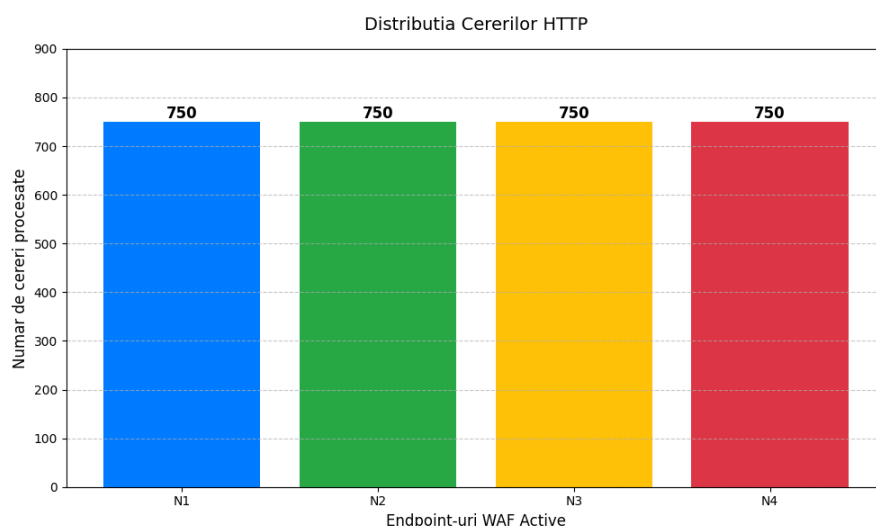


Figura 4.2. Grafic de distribuție a cererilor la nivel PoP între instanțele WAF active

Conform datelor ilustrate în graficul de mai sus, algoritmul Round-Robin distribuie traficul HTTP într-un mod perfect egal între toate nodurile de securitate disponibile. Această uniformitate confirmă faptul că starea endpoint-urilor (gestionată prin Redis) este corect gestionată, pregătind redirectionarea cererilor pentru evaluarea cu filtrele DPI (Deep Packet Inspection).

4.3. Testarea funcțională a securității: WAF și protecția volumetrică

După validarea mecanismelor de distribuire a traficului, următoarea etapă a constat în evaluarea directă a modului Web Application Firewall (WAF) și a legăturii acestuia cu rețeaua CDN în fața atacurilor cibernetice. Obiectivul acestor teste a fost să demonstreze eficiența filtrelor de tip rate limiting și a inspecției de profunzime a pachetelor (împotriva atacurilor calitative).

4.3.1. Protecția la nivelul Resolver-ului DNS (L3/L4)

Primul strat de securitate testat a fost mecanismul anti-flood implementat direct în DNS Resolver, înainte ca traficul să atingă stratul HTTP. A fost conceput un scenariu de atac în care un client a generat un trafic continuu de 240 de cereri pe secundă pe o durată de 10 secunde, vizând un domeniu gestionat de platformă. Limita fluxului de siguranță configurată pe server a fost de 80 de cereri pe secundă, cu un timp de penalizare (ban time) de 1 secundă.

Rezultatele au confirmat că resolver-ul a răspuns exclusiv la cererile care s-au încadrat în fereastra de trafic permis, după care mecanismele de logging au înregistrat depășirea pragului și au declanșat refuzul pachetelor (Drop). Această decuplare rapidă protejează pool-urile de adrese din spatele DNS-ului de epuizarea resurselor.

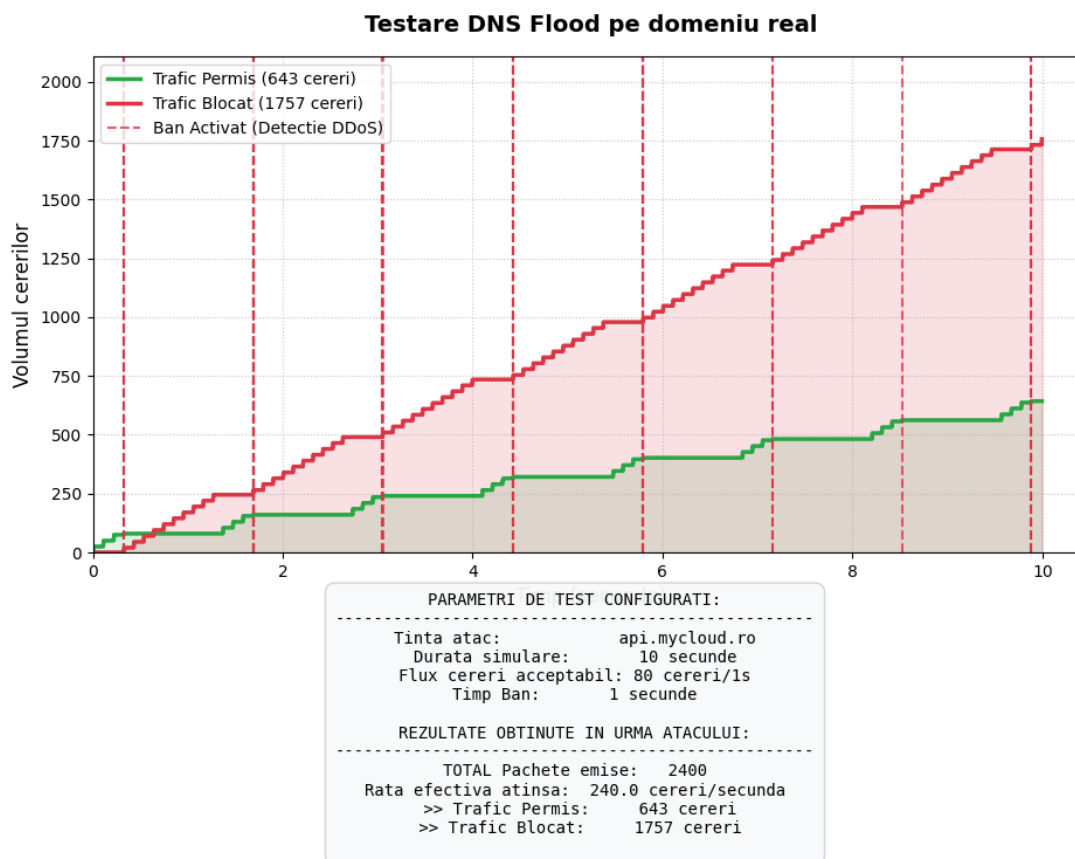


Figura 4.3. Evoluția traficului DNS în timpul simulării unui atac volumetric (DNS Flood)

4.3.2. Inspecția profundă a traficului On Ramp (L7 Signatures)

Al doilea experiment a vizat capacitatea sistemului de a detecta amenințări calitative ascunde în traficul on ramp. Pentru acest test, s-a restricționat intenționat rata de trimitere a pachetelor la sub 80 de cereri pe secundă, cu scopul de a „ocoli” filtrul de protecție volumetrică și de a forța modulul WAF să analizeze conținutul fiecărui request.

Au fost trimise zeci de cereri ce au conținut atât trafic legitim (CLEAN), cât și payload-uri malițioase definite în dicționarul OWASP al aplicației (SQL Injection, XSS, Path Traversal, accesarea porturilor sensibile și utilizarea de scannere cunoscute). WAF-ul a acționat ca un zid: 100% din cererile malițioase au fost respinse cu eroarea HTTP 403 Forbidden, în timp ce traficul CLEAN a fost permis fără întârzieri notabile. Graficul de distribuție evidențiază, de asemenea, că load balancing-ul pe endpoints a funcționat simultan, sarcinile de inspecție fiind împărțite în mod egal între toate containerele WAF active.

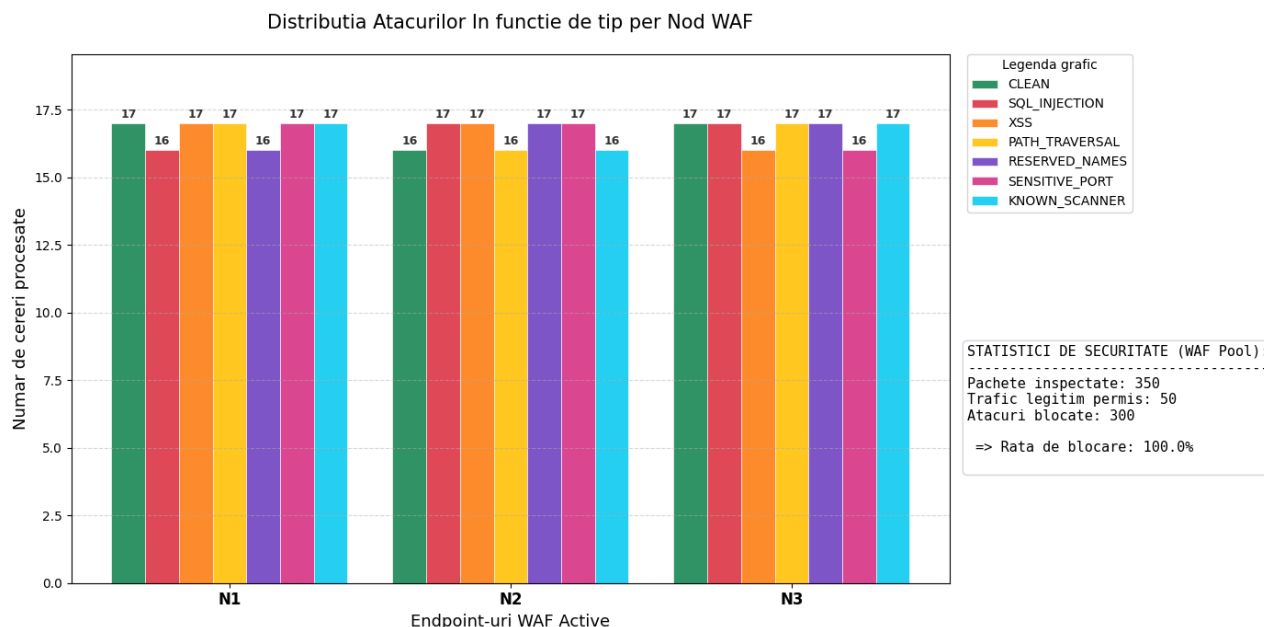


Figura 4.4. Distribuția tipurilor de atacuri detectate și a traficului legitim per nod WAF

4.3.3. Impactul WAF - CDN în atenuarea atacurilor HTTP Flood

Ultimul test de securitate a vizat comportamentul platformei sub presiunea unui atac HTTP Flood susținut, îndreptat către o resursă statică (un fișier imagine cu un TTL de 5 secunde). Acest experiment de 35 de secunde a demonstrat excelenta colaborare dintre filtrele de securitate și politicile de retenție a datelor. Pragul de detecție a unui atac din partea unei adrese a fost setat la 50 cereri/s.

Dinamica atacului a evidențiat un ciclu repetitiv, esențial pentru conservarea resurselor serverului de origine (off ramp):

1. Prima cerere primită generează un Cache Miss, solicitând resursa de la serverul de origine și stocând-o în memoria rețelei CDN.
2. Următoarele 49 de cereri ale atacatorului sunt servite instantaneu din memoria RAM a CDN-ului (Cache Hit), protejând serverul originar de prelucrare redundantă.
3. În secunda imediat următoare, pragul limitatorului volumetric din WAF este încălcat, iar WAF-ul activează penalizarea de 15 secunde (Ban). În acest interval, toate pachetele on ramp sunt respinse direct la nivel de PoP cu eroarea HTTP 429 Too Many Requests.
4. După expirarea ban-ului, ciclul se reia (TTL-ul resursei expirând între timp).

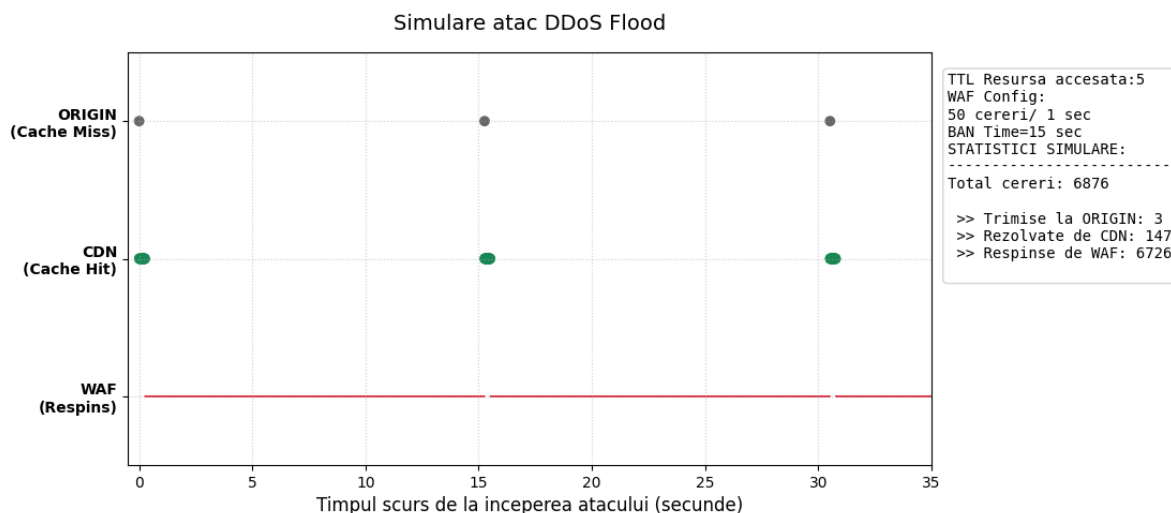


Figura 4.5. Atenuarea unui atac HTTP Flood prin acțiunea combinată a modulelor WAF și CDN

Concluzia acestui experiment este notabilă din punct de vedere arhitectural: pe toată durata celor 35 de secunde de stres intens, serverul de origine a fost accesat de doar 3 ori. Restul pachetelor au fost atenuate fie prin servirea datelor reținute temporar, fie prin blocarea proactivă a atacatorului. Acest mecanism asigură un acces concurrent optim pentru restul utilizatorilor legitimi, demonstrând o izolare perfectă a rutelor off ramp.

4.4. Fiabilitate, Failover și Limitările sistemului

O platformă cloud destinată prevenției DDoS trebuie să fie nu doar capabilă să filtreze traficul malițios, ci să aibă și un nivel ridicat de disponibilitate (High Availability) în cazul unor defecțiuni interne. Căderile pot surveni din cauza suprasolicitării hardware, a erorilor de procesare (software crashes) sau a pierderii conectivității izolate. Pentru a măsura reziliența arhitecturii, au fost concepute două scenarii de testare la stres care forțează oprirea nodurilor WAF din spatele Point of Presence.

4.4.1. Recuperarea la căderea totală a pool-ului WAF (Failover)

Primul test a vizat măsurarea timpului de recuperare (Recovery Time) în eventualitatea unui colaps simultan al tuturor nodurilor de securitate. Timp de 15 secunde, sistemul a fost supus unui trafic on ramp dens (utilizând 50 de thread-uri concurente). La secunda 5 a simulării, un script paralel a emis un semnal de oprire forțată către toate instanțele WAF, simulând o avarie critică.

Pe graficul rezultat se observă o întrerupere imediată a serviciului, moment în care cererile HTTP au început să returneze erori de conexiune (Timeout), devenind pachete pierdute (dropped). Platforma a demonstrat o capacitate automată de self-healing: orchestratorul Docker a repornit containerele afectate, iar infrastructura internă a PoP, prin intermediul clusterului Redis Local, a restabilit starea endpoint-urilor (Health Check). Timpul scurs de la prima eroare până la momentul revenirii traficului legitim definește eficiența mecanismului de failover.

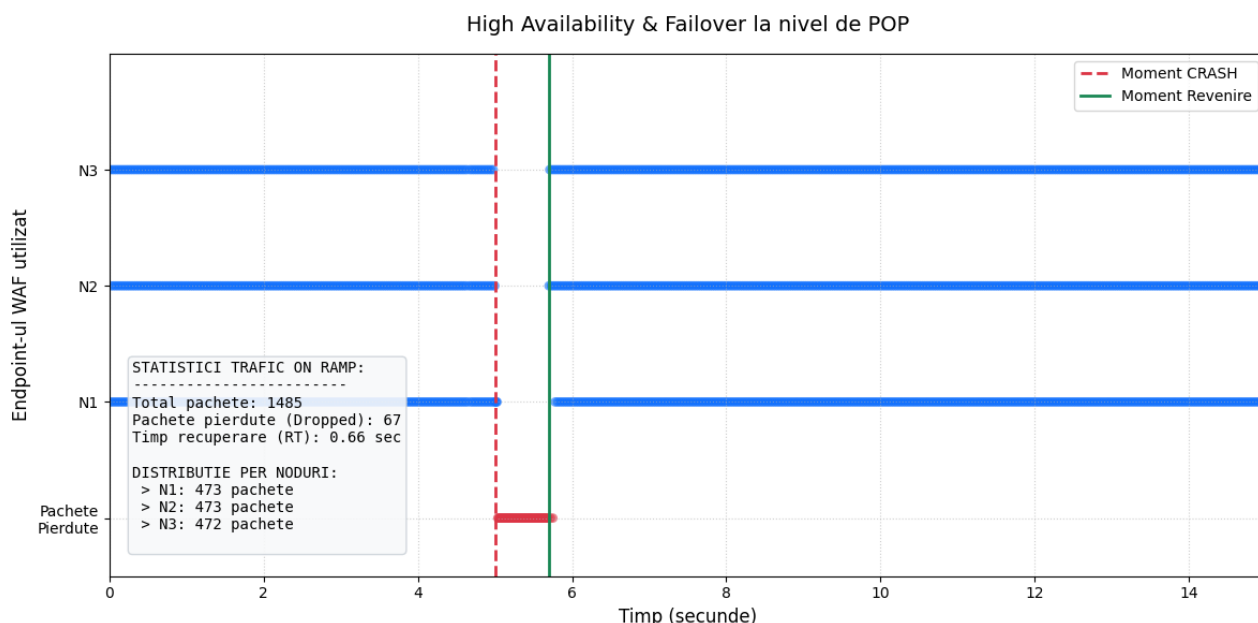


Figura 4.6. Reziliența Punctului de Prezență și timpul de recuperare la căderea simultană a nodurilor WAF

Rezultatele sunt foarte bune, înregistrându-se o medie de 22 de cereri pierdute per WAF în mai puțin de o secundă (≈ 0.66 s), acesta fiind și timpul după care serviciile de firewall au redevenit funcționale.

4.4.2. Eficiența mecanismului de detecție (Health Checks)

Al doilea experiment s-a axat pe dinamica de excludere a nodurilor unhealthy din pool-ul de Load Balancing. Într-o arhitectură scalată la 11 noduri WAF, pe parcursul a 90 de secunde, instanțele au fost oprite pe rând, la intervale aleatoare (între 0.5 și 5 secunde), până când a rămas un singur endpoint activ. Criteriile de detecție a flood-ului au fost din nou diminuate pentru a permite un trafic continuu necesar măsurării ferestrelor de vulnerabilitate.

Datele experimentale relevă ca „fereastră de detecție” se situează la ≈ 2 secunde, intervalul scurs din momentul în care un nod „moare” și până când PoP-ul își actualizează tabela de rutare din Redis pentru a-l elimina din algoritmul Round-Robin. În această scurtă perioadă, cererile rutate către nodul defect sunt pierdute. Timpul mediu de detecție a variat în funcție de latența rețelei interne, însă sistemul a reușit constant să izoleze incidentul și să stabilizeze traficul, distribuindu-l către nodurile rămase.

4.4.3. Limitări arhitecturale

Deși testele demonstrează o capacitate robustă de recuperare, platforma prezintă limitări inerente oricărui sistem de acest tip:

1. Pierderea de pachete în ferestrele de tranziție: Din cauza naturii asincrone a verificărilor de stare (Health Checks), există întotdeauna un număr de cereri on ramp care vor eșua înainte ca un nod să fie marcat ca inactiv. Acest comportament reprezintă un compromis între viteza de detecție și consumul de resurse al procesului de monitorizare.
2. Saturarea stratului fizic de rețea: Protecția oferită este eficientă la straturile L3/L4 (DNS) și L7 (HTTP). Cu toate acestea, într-un scenariu de DDoS volumetric extrem de masiv (ex: atacuri de sute de Gbps), capacitatea link-ului fizic al ISP-ului care găzduiește Points of Presence poate fi saturată înainte ca traficul să atingă stratul logic de load balancing, necesitând intervenții hardware la nivel de infrastructură upstream.

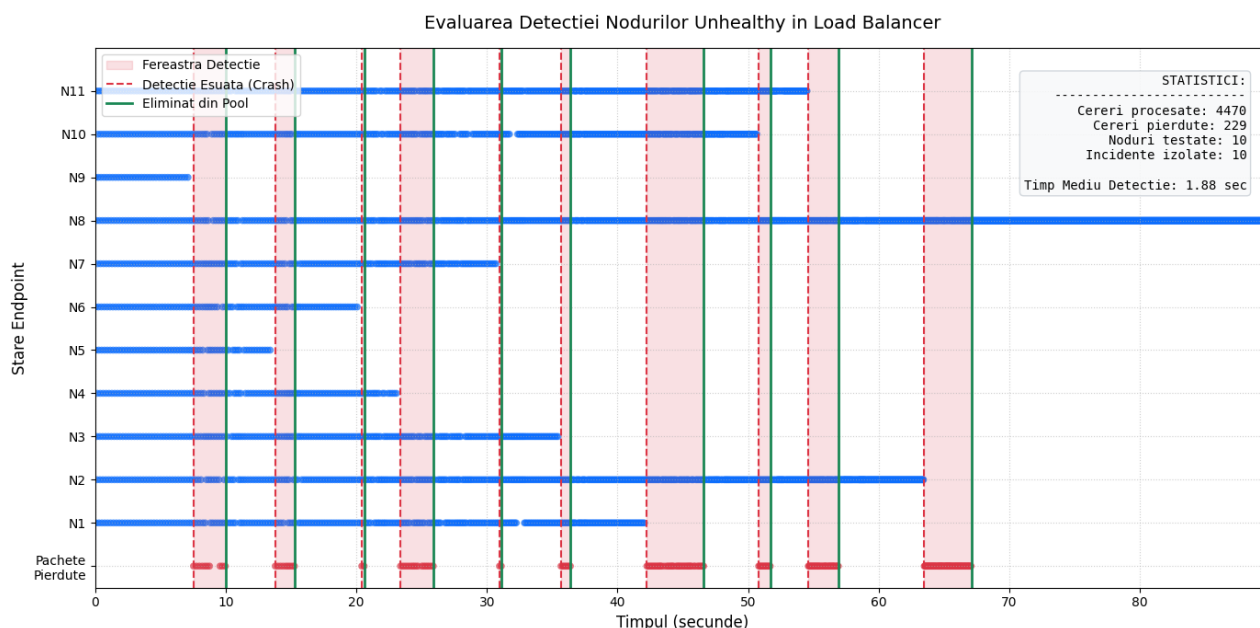


Figura 4.7. Evaluarea ferestrelor de detecție a nodurilor inactive și impactul asupra traficului On Ramp

4.5. Evaluarea performanței End-to-End (E2E)

Pentru a concluziona seria de experimente, a fost orchestrată o simulare complexă de tip End-to-End (E2E), menită să evalueze comportamentul platformei în ansamblu. Acest test a supus întregul pipeline de procesare (DNS -> On Ramp -> WAF -> CDN -> Off Ramp) la o presiune mixtă, rulând o succesiune de patru scenarii distincte și simultane:

1. Crowd (Trafic Legitim Susținut)

S-a generat un volum ridicat de cereri concurente valide, simulând o creștere bruscă, dar organică, a popularității resursei. Sistemul a făcut față cu succes, traficul fiind absorbit de memoria cache a modului CDN, protejând serverul de origine de o potențială suprasolicitare.

2. DNS Amplification (Atac L3/L4 pe Resolver)

Un flux de interogări a vizat direct resolver-ul global. Mecanismul de rate limiting a intervenit, rezolvând un număr strict de cereri valide și aplicând acțiunea de „Drop” (prin timeout) pachetelor malițioase. Această decizie la marginea rețelei a prevenit propagarea atacului de amplificare către infrastructura Points of Presence.

3. HTTP Flood (Atac L7 Volumetric)

Zeci de fire de execuție (workers) au atacat PoP-ul cu trafic HTTP on ramp. Modulul WAF a detectat rapid încălcarea pragului de flood la nivel de aplicație, respingând fluxul de pachete și răspunzând cu codul de eroare 429 Too Many Requests, păstrând în același timp lățimea de bandă și ciclul de procesare pentru traficul legitim.

4. L7 Signatures (Atac Calitativ)

Printre pachetele legitime au fost injectate payload-uri malițioase (SQL Injection și XSS) la o rată de trimitere redusă, menită să evite declanșarea protecției volumetrice. Prin mecanismul de Deep Packet Inspection, nodurile WAF au identificat și blocat exclusiv aceste încercări (Eroare HTTP 403 Forbidden). Execuția acestui scenariu a validat faptul că load balancing-ul pe endpoints distribuie echitabil efortul de inspecție pe tot pool-ul WAF.

Rezultatele acestei testări agregate au fost centralizate într-un tablou de bord (Dashboard) E2E, care ilustrează transparent parcursul pachetelor de-a lungul straturilor OSI și deciziile luate la fiecare nod de rețea.

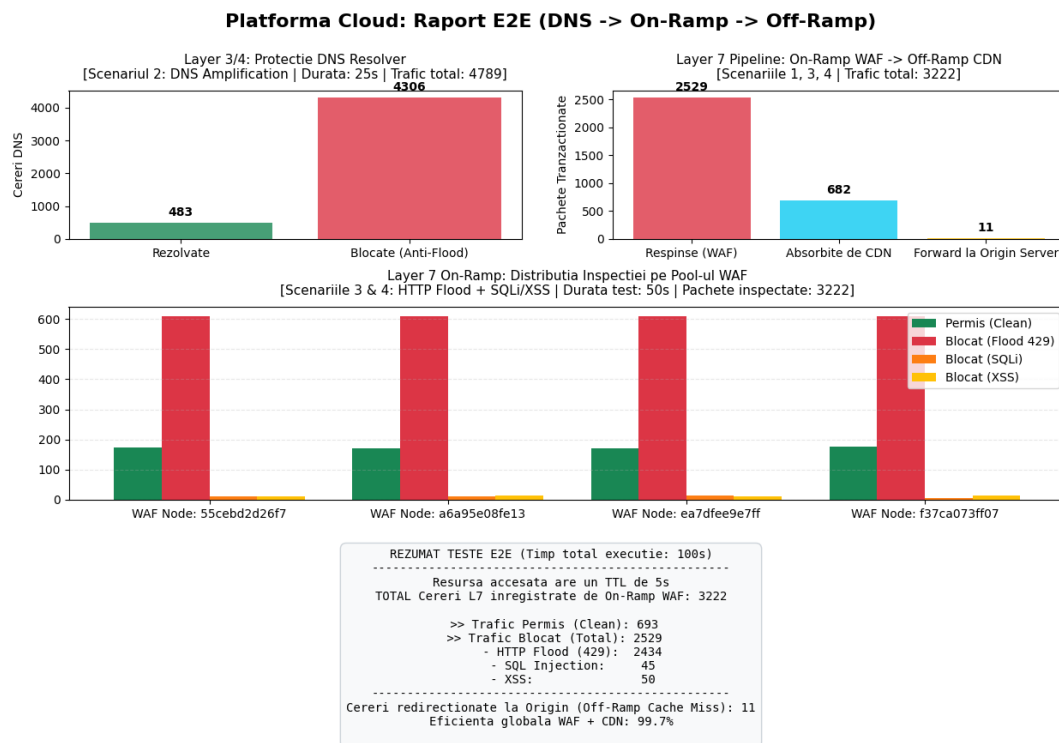


Figura 4.8. Colecția de date E2E ilustrând performanța mecanismelor de protecție L3 - L7 și distribuția traficului pe parcursul celor 4 scenarii mixte

4.5.1. Analiza Eficienței Globale (Concluzia experimentelor)

Cel mai relevant indicator de performanță rezultat din această simulare este metrica de „Eficiență globală WAF + CDN”. Din miile de pachete on ramp generate – cuprinzând trafic malițios volumetric, atacuri calitative specifice și trafic legitim intens – doar o mică fracțiune (reprezentată de starea de Origin Miss, determinată exclusiv de expirarea regulată a parametrului TTL) a ajuns să consume resursele de calcul ale serverului originar.

Platforma a înregistrat o eficiență de izolare a rutei off ramp superioară, blocând proactiv atacurile cibernetice și servind solicitările valide dintr-o memorie distribuită. Aceste rezultate experimentale confirmă atingerea tuturor obiectivelor arhitecturale propuse: sistemul cloud dezvoltat oferă o prevenție robustă și demonstrabilă împotriva atacurilor DDoS, dispune de mecanisme avansate de logare, asigură scalabilitatea serviciilor prin load balancing și menține o disponibilitate ridicată pentru utilizatorii finali.

Capitolul 5. Concluzii

5.1. Realizarea obiectivelor și contribuții personale

Prezenta lucrare a urmărit și a îndeplinit cu succes obiectivul principal de a dezvolta o platformă cloud modulară și robustă, destinată prevenției atacurilor DDoS și optimizării livrării de conținut web. Pornind de la premisele stabilite în capitolele introductive, arhitectura a fost implementată respectând principiile moderne de rețea și containerizare, atingând astfel toate ipotezele tehnic stabilite.

Gradul de realizare a temei este unul complet, fiind integrate următoarele module și funcționalități critice:

- S-a proiectat și implementat un DNS Resolver global, capabil să rezolve interogările pentru diverse TLD-uri și să aplice un mecanism de load balancing pe pools pentru a distribui traficul încă de la nivelul L3/L4 OSI.
- S-au dezvoltat module de tip PoP (Point of Presence) care interceptează traficul on ramp și aplică eficient un mecanism de load balancing pe endpoints către nodurile de securitate din spate.
- A fost integrat un modul WAF (Web Application Firewall) avansat, echipat cu detecția volumului cererilor și inspecție profundă a pachetelor (DPI) pentru semnăturile OWASP.
- S-a construit o rețea CDN susținută de un sistem de stocare Redis, care absoarbe șocurile de trafic, asigurând atât o latență minimă, cât și o izolare perfectă a traficului off ramp către serverul de origine.

Din punct de vedere al contribuțiilor personale, se remarcă efortul de integrare a acestor componente într-un pipeline End-to-End perfect funcțional. S-au dezvoltat mecanisme complexe de tip health check pentru a garanta o disponibilitate ridicată (High Availability) în caz de avarii ale nodurilor, alături de security and logging mechanisms detaliate care au permis monitorizarea și extragerea datelor experimentale. Testele de stres au demonstrat clar că platforma asigură scalabilitate orizontală și oferă un acces concurent excelent pentru utilizatorii legitimi, chiar și sub presiunea atacurilor simulate.

5.2. Comparație cu alte proiecte și soluții similare

Pentru a valida soluția propusă, este necesară o paralelă cu liderii industriei de securitate cibernetică, în mod special cu platforma Cloudflare, care a servit drept model arhitectural de bază pentru acest proiect.

Platformele comerciale de calibru, precum Cloudflare sau AWS Shield, se bazează pe o infrastructură fizică globală masivă, utilizând rutare anycast reală la nivel de hardware și rețele interconectate prin protocoale BGP. În contrast, proiectul de față a reprodus această logică într-un mediu virtualizat și strict controlat, alocarea dinamică a resurselor și a IP-urilor (echivalentul unui serviciu DHCP intern) făcându-se prin intermediul orchestratorului Docker.

Spre deosebire de soluțiile comerciale unde logica de filtrare este de tip „black-box” pentru utilizatorul final, platforma dezvoltată în această lucrare oferă transparență totală asupra fluxului de date. Traseul exact al pachetului – de la interogarea la DNS Resolver, intrarea în PoP (on ramp), inspecția în WAF, decizia din CDN și eventuala trimitere spre origine (off ramp) – poate fi monitorizat și modificat la nivel de cod sursă. De asemenea, față de soluțiile open-source izolate (precum instalarea unui simplu NGINX cu ModSecurity), acest sistem reprezintă un întreg ecosistem cloud

sincronizat. Principalul neajuns al soluției dezvoltate, în comparație cu giganții din industrie, îl reprezintă limitarea la capacitatea stratului fizic de rețea pe care rulează, platforma protejând eficient la nivel L7, dar depinzând de lățimea de bandă a mașinii gazdă în cazul unui atac volumetric brut.

5.3. Posibile direcții de dezvoltare

Deși platforma oferă în stadiul actual o protecție robustă și demonstrabilă, arhitectura sa modulară permite o serie de extinderi și optimizări tehnice pe viitor. Au fost identificate următoarele direcții principale de dezvoltare:

- **Monitorizare și telemetrie în timp real:** Înlocuirea procesului de generare statică a graficelor de performanță (post-testare) cu o soluție de generare dinamică la run-time.
- **Decuplarea responsabilităților arhitecturale:** În prezent, serviciul PoP conține și logica de auto-înregistrare în DNS. O direcție viitoare este extragerea acestei funcții într-un microserviciu separat de tip „Service Discovery”, respectând astfel principiul separării responsabilităților.
- **Politici API dinamice (Multi-tenancy):** Implementarea unui panou de control prin care fiecare client al platformei să poată configura o „API policy” dinamică, ajustând independent pragurile de rate limiting și seturile de semnături WAF strict pentru domeniul propriu, fără a afecta alți utilizatori.
- **Load balancing pe rutele off ramp:** Momentan, sistemul extrage datele de la o singură adresă de origine în cazul unui cache miss. Implementarea unui mecanism de load balancing și pentru traficul de ieșire (off ramp) ar permite clienților să aibă propriul lor cluster de servere în spate, prevenind blocajele.
- **Arhitectura Shared CDN:** Optimizarea stocării prin interconectarea memoriilor cache între diferitele Puncte de Prezență (Pop). Într-un astfel de scenariu, dacă un PoP înregistrează un cache miss, acesta va interoga mai întâi sistemele CDN din celelalte PoP-uri înainte de a solicita datele de la serverul de origine, minimizând și mai mult traficul off ramp global.

5.4. Lecții învățate pe parcursul dezvoltării proiectului de diplomă

Proiectarea mediului a presupus informarea cu privire la standardele din industrie și adaptarea elementelor utile în cadrul implementării modulelor DNS, PoP, WAF și CDN. Acest lucru a implicat înțelegerea standardelor de scriere a documentațiilor, diferențierea dintre reguli, elementele de bune practici și cele opționale.

De asemenea, în cadrul dezvoltării aplicației propuse, am fost nevoit să prioritizez țintele comportamental-arhitecturale și să găsesc soluții de implementare care să mențină o cuplare cât mai slabă între module, păstrând caracterul parametrizabil al serviciilor.

În faza testelor, am fost pus în situația de a agrega setul de date relevant în formularea statisticilor și reprezentarea grafică explicită a rezultatelor obținute.

Și nu în ultimul rând, scrierea documentației a făcut obiectul unui efort de aliniere la regulile de expunere ale unui text academic, respectând rigorile științifice și standardele de redactare.

Bibliografie

- [1] P. Mockapetris, “Domain Names - Concepts and Facilities,” RFC 1034, 1987. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc1034>
- [2] —, “Domain Names - Implementation and Specification,” RFC 1035, 1987. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc1035>
- [3] T. Berners-Lee, R. Fielding, and L. Masinter, “Uniform Resource Identifier (URI): Generic Syntax,” RFC 3986, 2005. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc3986>
- [4] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Caching,” RFC 9111, 2022. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc9111>
- [5] M. Nottingham, “HTTP Cache-Control Extensions for Stale Content,” RFC 5861, 2010. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc5861>
- [6] A. Peters and M. Nilsson, “Forwarded HTTP Extension,” RFC 7239, 2014. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7239>
- [7] Cloudflare, “What is a DNS Amplification Attack?” <https://www.cloudflare.com/learning/ddos/dns-amplification-ddos-attack/>, 2026, Ultima accesare: 04.06.2026.
- [8] J. Abley, O. Gudmundsson, M. Majkowski, and E. Hunt, “Providing Minimal-Sized Responses to DNS Queries That Have QTYPE=ANY,” RFC 8482, 2019. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8482>
- [9] Cloudflare, “What is the OSI Model?” <https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/>, 2026, Ultima accesare: 04.06.2026.
- [10] OWASP, “ModSecurity Core Rule Set (CRS),” <https://coreruleset.org/>, 2026, Ultima accesare: 04.06.2026.
- [11] —, “A05:2025 - Injection,” https://owasp.org/Top10/2025/A05_2025-Injection/, 2025, Ultima accesare: 04.06.2026.
- [12] Cloudflare, “What is a Slowloris DDoS attack?” <https://www.cloudflare.com/learning/ddos/ddos-attack-tools/slowloris/>, 2026, Ultima accesare: 25.06.2026.
- [13] —, “Load Balancing Architecture Reference,” <https://developers.cloudflare.com/reference-architecture/architectures/load-balancing/>, 2026, Ultima accesare: 04.06.2026.
- [14] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” RFC 9110, 2022. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc9110>
- [15] Cloudflare, “Docs directory,” <https://developers.cloudflare.com/directory/?product-group=Docs+collections>, 2026, Ultima accesare: 04.06.2026.
- [16] —, “Reference Arhitecture,” <https://developers.cloudflare.com/reference-architecture/>, 2026, Ultima accesare: 04.06.2026.
- [17] —, “What is Anycast?” <https://www.cloudflare.com/learning/cdn/glossary/anycast-network/>, 2026, Ultima accesare: 04.06.2026.

- [18] Docker, “Docker Docs,” <https://docs.docker.com/reference/>, 2026, Ultima accesare: 04.06.2026.
- [19] Redis, “Redis Docs,” <https://redis.io/docs/latest/>, 2026, Ultima accesare: 04.06.2026.

Anexe

În această anexa se regăsesc fișierele de configurare și implementarea software completă pentru modulele dezvoltate.

Anexa 1. Topologia Rețelei si Serviciile Containerizate din DNS

Configurația completă a containerelor Docker pentru emularea ierarhiei DNS si managementul platformei este descrisă în fișierul de mai jos.

Listing 1: Fișierul docker-compose.yml pentru containerul DNS

```

1  version: '3.8'
2
3  networks:
4    dns_network:
5      driver: bridge
6    mycloud_global_net:
7      external: true
8
9  services:
10   redis_root:
11     image: redis:latest
12     container_name: redis_root
13     networks:
14       - dns_network
15
16   ns_root:
17     build: ./DNS_Nameserver
18     container_name: ns_root
19     depends_on:
20       - redis_root
21     environment:
22       - REDIS_HOST=redis_root
23       - REDIS_PORT=6379
24       - NS_NAME=ROOT_SERVER
25       - NS_ROLE=ROOT
26       - IP_BIND=0.0.0.0
27       - PORT_BIND=53
28       - ROTLD_HOSTNAME=ns_rotld
29     networks:
30       - dns_network
31
32   redis_rotld:
33     image: redis:latest
34     container_name: redis_rotld
35     networks:
36       - dns_network
37
38   ns_rotld:
39     build: ./DNS_Nameserver
40     container_name: ns_rotld

```

```
41     depends_on:
42         - redis_rotld
43     environment:
44         - REDIS_HOST=redis_rotld
45         - REDIS_PORT=6379
46         - NS_NAME=ROTLD_SERVER
47         - NS_ROLE=ROTLD
48         - IP_BIND=0.0.0.0
49         - PORT_BIND=53
50         - MYCLOUD_HOSTNAME=ns_mycloud
51     networks:
52         - dns_network
53
54     redis_mycloud:
55         image: redis:latest
56         container_name: redis_mycloud
57         networks:
58             - dns_network
59             - mycloud_global_net
60
61     mycloud_control_plane:
62         build: ./DNS_Nameserver
63         container_name: control_plane_dns
64         command: [ "python", "-u", "control_plane.py" ]
65         depends_on:
66             - redis_mycloud
67         environment:
68             - REDIS_HOST=redis_mycloud
69         networks:
70             - dns_network
71
72     ns_mycloud:
73         build: ./DNS_Nameserver
74         container_name: ns_mycloud
75         depends_on:
76             - redis_mycloud
77         environment:
78             - REDIS_HOST=redis_mycloud
79             - REDIS_PORT=6379
80             - NS_NAME=MYCLOUD_AUTH
81             - NS_ROLE=MYCLOUD
82             - IP_BIND=0.0.0.0
83             - PORT_BIND=53
84         networks:
85             - dns_network
86
87     redis_resolver:
88         image: redis:latest
89         container_name: redis_resolver
90         networks:
91             - dns_network
92
93     resolver_app:
94         build: ./DNS_Resolver
```

```

95     container_name: dns_resolver_NO_1
96     depends_on:
97         - ns_root
98         - redis_resolver
99     environment:
100         - RESOLVER_NAME=DNS_Resolver_N1
101         - IP_BIND=0.0.0.0
102         - PORT_BIND=5333
103         - ROOT_HOSTNAME=ns_root
104         - NS_TARGET_PORT=53
105         - REDIS_HOST=redis_resolver
106         - REDIS_PORT=6379
107         - DNS_FLOOD_WINDOW=1
108         - DNS_FLOOD_MAX_REQS=80 #pune 800000
109         - DNS_BAN_TIMEOUT=15
110         - NXDOMAIN_CACHE_TTL=10
111     ports:
112         - "5333:5333/udp"
113     networks:
114         - dns_network
115
116 rotld_api:
117     build: ./MyCloud_Manager
118     container_name: rotld_api
119     command: [ "python", "-u", "rotld_api.py" ]
120     ports:
121         - "8080:8080"
122     environment:
123         - ROTLD_REDIS_HOST=redis_rotld
124         - ROTLD_PORT=8080
125     depends_on:
126         - redis_rotld
127     networks:
128         - dns_network
129
130 mycloud_api:
131     build: ./MyCloud_Manager
132     container_name: mycloud_api
133     command: [ "python", "-u", "myCloud_api.py" ]
134     ports:
135         - "5050:5000"
136     environment:
137         - MYCLOUD_REDIS_HOST=redis_mycloud
138         - API_PORT=5000
139         - API_KEY=micunealta-secreta
140         - ROTLD_API_URL=http://rotld_api:8080
141     depends_on:
142         - redis_mycloud
143         - rotld_api
144     networks:
145         - dns_network

```

Anexa 2. Codul funcției `get_nearest_ancestor`

Listing 2: Codul funcției `get_nearest_ancestor`

```

1  def get_nearest_ancestor(qname):
2      root_ip = SBelt["root"]["ips"][0]
3
4      qname = qname.lower()
5
6      # verific daca un ancestor a mai fost cautat si s-a primit o
        ↳ referinta a unui NS
7      labels = [l for l in qname.split('.') if l]
8      search_names = [ ".".join(labels[i:]) + "." for i in
        ↳ range(len(labels))]
9      search_names.append(".")
10
11     for name in search_names:
12         cache_key = f"NS:{name}" # cautam NS urile asociate
13         record_json = db.get(cache_key)
14         record_ttl=db.ttl(cache_key)
15         if record_json:
16             record = json.loads(record_json)
17             if record.get('type') == 'NS':
18                 print(f"[I]: ~CACHE NS HIT~ {RESOLVER_NAME} a gasit
                    ↳ nearest ancestor pt {qname} -> {name} (TTL:
                    ↳ {record_ttl})")
19                 returned_list = list(record.get('ips', []))
20                 returned_list.append(root_ip) # pentru orice
                    ↳ eventualitate, cname uri care nu au primit si o
                    ↳ lista de adrese efective, etc. Punem si adresa root
                    ↳ ului
21                 return returned_list
22
23     print(f"[I]: {RESOLVER_NAME} Nu a gasit delegari. Getting data from
        ↳ SBelt (Root):")
24     return SBelt["root"]["ips"]

```

Anexa 3. Codul modulului de identificare a PoP-urilor active

Listing 3: Codul funcției de identificare al PoP-urilor active

```

1  def actualizeaza_rute_dns():
2      while True:
3          try:
4              chei_active = db.keys("heartbeat:POP:*")
5              ips_live = [k.split(":")[-1] for k in chei_active]
6
7              if ips_live:
8                  for domeniu in DOMENII_PROTEJATE:
9                      date_dns = {
10                          "type": "A",

```

```

11         "ips": ips_live,
12         "ttl": 30
13     }
14     db.set(f"A:{domeniu}", json.dumps(date_dns))
15     else:
16         for domeniu in DOMENII_PROTEJATE:
17             db.delete(f"A:{domeniu}")
18
19     except Exception as e:
20         print(f"[*E] Eroare in Control Plane: {e}")
21
22     time.sleep(5)

```

Anexa 4. Topologia Rețelei și Serviciile Containerizate din PoP

Configurația completă a containerelor Docker pentru emularea PoP managementul platformei este descrisă în fișierul de mai jos.

Listing 4: Fișierul docker-compose.yml pentru containerul POP

```

1  version: '3.8'
2
3  services:
4      redis:
5          image: redis:alpine
6          networks:
7              - ddos_net
8
9      pop_lb:
10         build: .
11         command: python main.py
12         ports:
13             - "8085-8090:80"
14         environment:
15             - POP_PORT=80
16             - REDIS_HOST=redis
17             - GLOBAL_DNS_REDIS=redis_mycloud
18             - PYTHONUNBUFFERED=1
19         depends_on:
20             - redis
21         networks:
22             - ddos_net
23             - mycloud_global_net
24
25      waf_node:
26         build: .
27         command: python waf.py 8000
28         restart: always
29         #restart: "no"
30         environment:
31             - REDIS_HOST=redis
32             - WAF_FLOOD_WINDOW=1
33             - WAF_FLOOD_MAX_REQS=50
34             - WAF_BAN_TIMEOUT=30

```

```
35         - PYTHONUNBUFFERED=1
36     depends_on:
37         - redis
38     networks:
39         - ddos_net
40         - mycloud_global_net
41
42
43 networks:
44     ddos_net:
45         driver: bridge
46     mycloud_global_net:
47         external: true
```

Anexa 5. Codul funcției de semnalare în platformă a PoP-urilor active

Listing 5: Codul serviciului de identificare al PoP-urilor active

```
1 def send_heartbeat_to_dns():
2     try:
3         dns_db = redis.Redis(host=GLOBAL_DNS_REDIS, port=6379,
4                               ↪ decode_responses=True)
5     except Exception as e:
6         print(f"[*E] Eroare la conexiunea cu DNS Global: {e}")
7         return
8
9     while True:
10        my_ip = get_my_global_ip()
11        if my_ip != '127.0.0.1':
12            try:
13                dns_db.set(f"heartbeat:POP:{my_ip}", "alive", ex=15)
14                print(f"[*I] [HEARTBEAT] POP la {my_ip} înregistrat în
15                      ↪ DNS.")
16            except Exception as e:
17                print(f"[*E] Heartbeat a esuat: {e}")
18        time.sleep(10)
```

Anexa 6. Codul clasei responsabile de redirectionare corectă a cererilor către WAFs

Listing 6: Logica proxy-ului transparent pentru rutarea traficului (On Ramp)

```
1 class ProxyHTTPRequestHandler(http.server.BaseHTTPRequestHandler):
2     def do_GET(self): self.handle_request("GET")
3     def do_POST(self): self.handle_request("POST")
4
5     def handle_request(self, method):
6         client_ip = self.client_address[0]
7         target_endpoint = lb_core.get_next_endpoint()
8
9         if not target_endpoint:
```



```

10         self.send_error(503, "Niciun endpoint WAF disponibil.")
11         return
12
13     target_url = f"{target_endpoint}{self.path}"
14     body_data = None
15
16     if method == "POST":
17         content_length = int(self.headers.get('Content-Length', 0))
18         if content_length > 0:
19             body_data = self.rfile.read(content_length)
20
21     try:
22         req = urllib.request.Request(target_url, data=body_data,
23                                     ↪ method=method)
24
25         # copiem headerele originale si adaugam IP-ul real al
26         ↪ clientului
27         for key, value in self.headers.items():
28             req.add_header(key, value)
29         req.add_header('X-Forwarded-For', client_ip)
30
31         # trimitem cererea catre WAF si intoarcem raspunsul
32         with urllib.request.urlopen(req, timeout=5) as response:
33             self.send_response(response.getcode())
34             for key, value in response.info().items():
35                 self.send_header(key, value)
36             self.end_headers()
37             self.wfile.write(response.read())
38
39     except urllib.error.HTTPError as e:
40         # Propagam erorile generate de securitatea WAF (ex: 403
41         ↪ Forbidden)
42         self.send_response(e.code)
43         for key, value in e.headers.items():
44             self.send_header(key, value)
45         self.end_headers()
46         self.wfile.write(e.read())
47
48     except urllib.error.URLError as e:
49         # Tratatam si cazul in care nodul WAF pica
50         self.send_error(502, "Bad Gateway: WAF endpoint
51         ↪ communication failed.")

```

Anexa 7. Codul funcției responsabile de înregistrarea WAF în rândul nodurilor active

Listing 7: Logica de auto-înscrisere în rândul nodurilor active

```

1 def register_to_redis():
2     my_hostname = socket.gethostname()
3     endpoint_url = f"http://{my_hostname}:{PORT}"
4
5     while True:
6         try:

```

```
7         # am pus ttl 10 secunde
8         redis_client.set(f"waf_node:{endpoint_url}", "online",
9                             → ex=10)
10    except Exception as e:
11        print(f"[*E] Redis error: {e}")
12    time.sleep(5)
```

Anexa 8. Dictionarul de semnături Regex

Listing 8: Dictionarul de semnături Regex pentru detecția atacurilor L7

```
1  # Semnături bazate pe standardele OWASP si RFC 3986
2  ATTACK_SIGNATURES = {
3      "SQL_INJECTION": re.compile(
4          r"(?i)(union.*select|insert.*into|drop\s+table
5              waitfor\s+delay|sleep\s*\(|information_schema|' OR \d+=\d+|'
6              OR '[a-z]'='[a-z]|--$)"
7      ),
8      "XSS": re.compile(r"(?i)(<script>|javascript:|onerror=)" ,
9      # Prevenire Path Traversal si acces fisiere sensibile (RFC 3986)
10     "PATH_TRAVERSAL":
11     re.compile(r"(?i)(\.\.\/|\.\.\\|/etc/passwd|\\x00)" ,
12     # Nume rezervate de sistem si acces porturi sensibile (RFC 3986)
13     "RESERVED_NAMES":
14     re.compile(r"(?i)\b(AUX|PRN|CON|LPT[1-9]|COM[1-9])\b)" ,
15     "SENSITIVE_PORT_ACCESS":
16     re.compile(r":([0-9]{1,3}|102[0-3])($|/|\?)" ,
17     # Blocare scannere cunoscute si boti (CWE-20)
18     "KNOWN_SCANNER":
19     re.compile(r"(?i)(sqlmap|nikto|wpscan|dirbuster|nmap|zgrab|masscan|python-recon-ng|
20     ) }
```